Data Structure

Haiyang Liu

Tree and Its Algorithm

Outline: Content

- The basic concept of a tree;
- The properties and storage structure of binary trees;
- Traverse the binary tree;
- Threaded binary tree
- The storage and traversal of the tree;
- Huffman Tree and Its applications

Main content

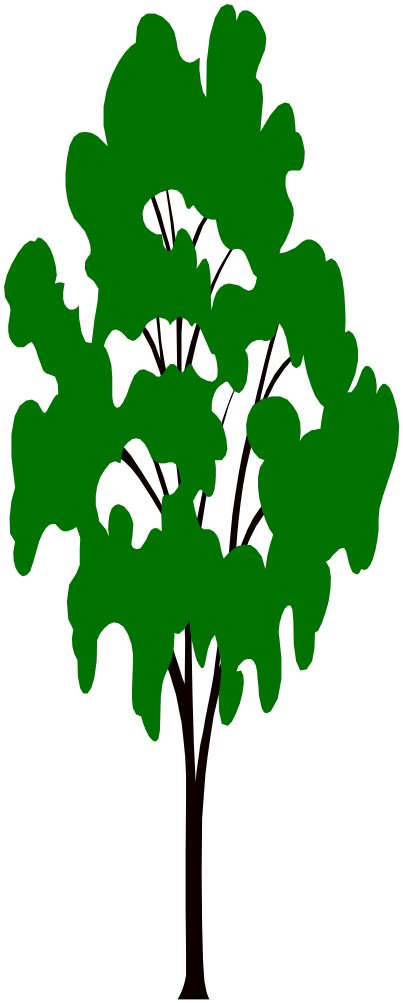
- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- Traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

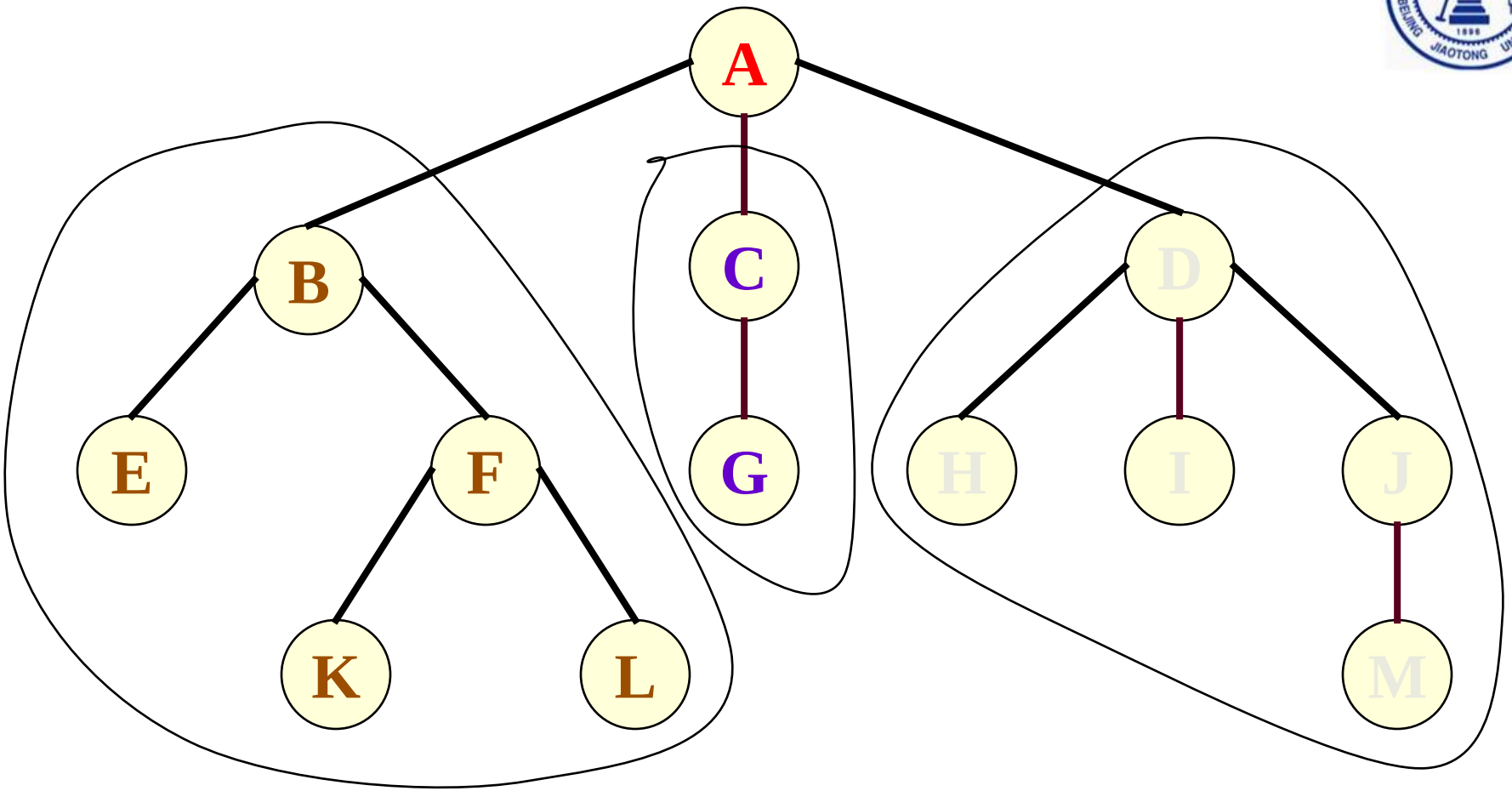
Main content

- **Tree type definitions**
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- Traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

The definition of a tree

- A tree is a finite set consisting of n nodes (where n ≥ 0). When n = 0, it is called an empty tree (recursion base). In any non-empty tree, there exists and only one node called the root. The remaining nodes can be divided into m disjoint finite sets T_1, T_2,..., T_m, each of which itself forms a tree known as a subtree of the root. Similarly, each subtree can be further divided into multiple disjoint finite sets.





A(B(E, F(K, L)), C(G), D(H, I, J(M)))

Root T_1 T_2 T_3

© Haiyang Liu, 2025

Tree as an abstract data type

ADT Tree{

Data object D: D is a collection of data elements with the same characteristics.

Data relationship R: If D is empty, it is called empty tree; otherwise:

(1) There exists a unique data element called root in D;

(2) When n>1, the remaining nodes can be divided into m (m>0) disjoint finite sets T_1, T_2,..., T_m, each of which itself is a tree conforming to this definition, called a subtree of the root root.

Basic operations: basic class, search class, insert class, delete class, etc.

} ADT Tree

Basic operations: searching

- Root(T) // Find the root node of a tree
- Value(T, cur_e) // Get the value of an element at the current node
- Parent(T, cur_e) //Returns the parent node of the current node
- LeftChild (T, cur_e) //Returns the leftmost child of the current node
- RightSibling (T, cur_e) // Find the right sibling of the current node
- TreeEmpty (T) // determines whether the tree is empty or not
- TreeDepth (T) // Calculate the depth of the tree
- TraverseTree (T, Visit()) // traversal

Basic operations: insertion

- InitTree(&T) // Initialize an empty tree
- CreateTree(&T, definition) // Construct the tree as defined
- Assign(T, cur_e, value) // Assigns a value to the current node
- InsertChild(&T, &p, i, c) // Inserts the tree with c as the root into the i-th subtree of the node p

© Haiyang Liu, 2025

Basic actions: deletion

- ClearTree(&T) // Clear the tree
- DestroyTree(&T) // Destroy the tree structure
- DeleteChild(&T, &p, i) // Delete the i-th subtree of node p

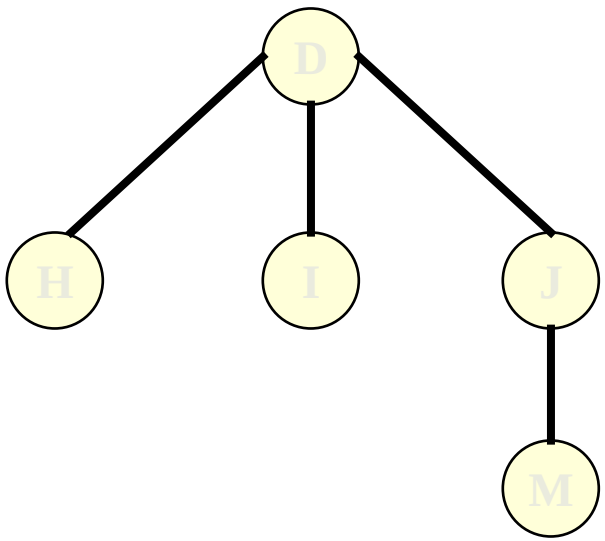
Tree structure vs Linear structure

linear structure	tree structure
One first data element (No precursor)	One root node (No precursor)
One last data element (No successor)	Multiple leaf nodes (No successor)
Other data elements (One precursor, one successor)	Other data elements (One precursor, Multiple successors)

© Haiyang Liu, 2025

Basic terms of trees (6-1)

- **Tree node:** contains a data element and several branches pointing to its subtrees.
- **Degree of a node:** the number of child trees (branches) a node has.

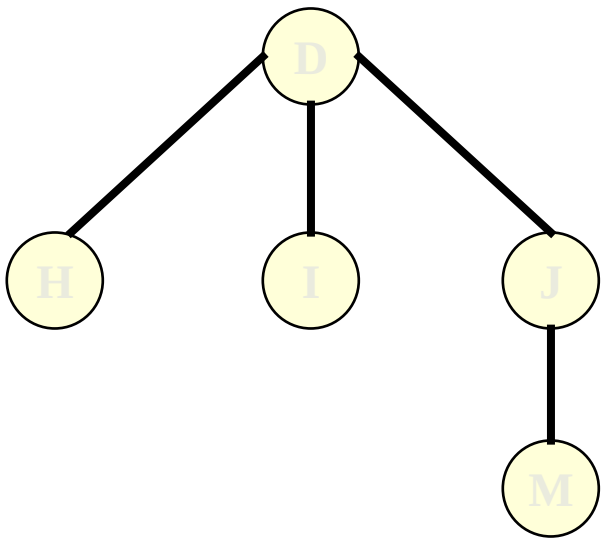


- **Tree degree:** the maximum degree of all nodes in a tree.
- **Leaf node :** a node with degree zero.
- **Branch node (non-leaf node):** a node with a degree greater than zero.

© Haiyang Liu, 2025

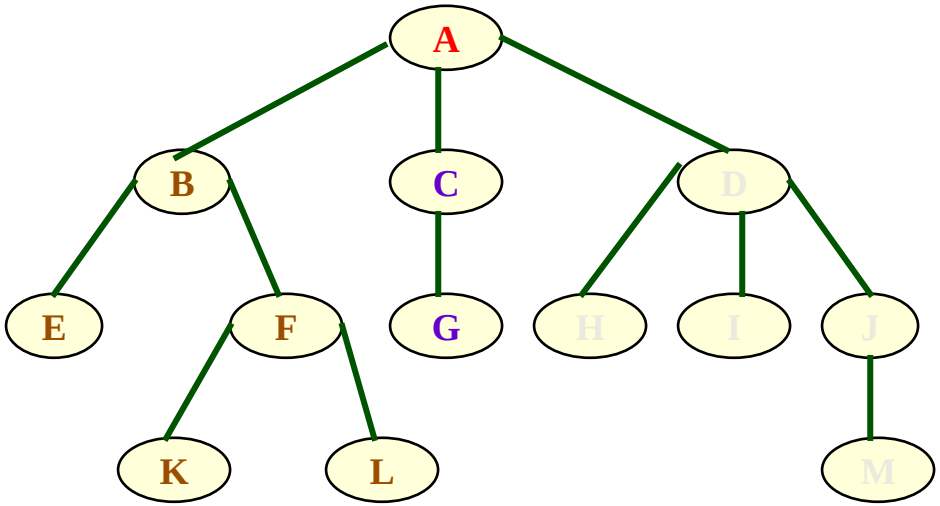
Basic Terms for Trees (6-2)

- A path (from root to node) consists of the branches and nodes traversed from the root to that node.
- Child node: the root of the child tree of a node.
- Parent node: accordingly, the node is called the child's parent node.



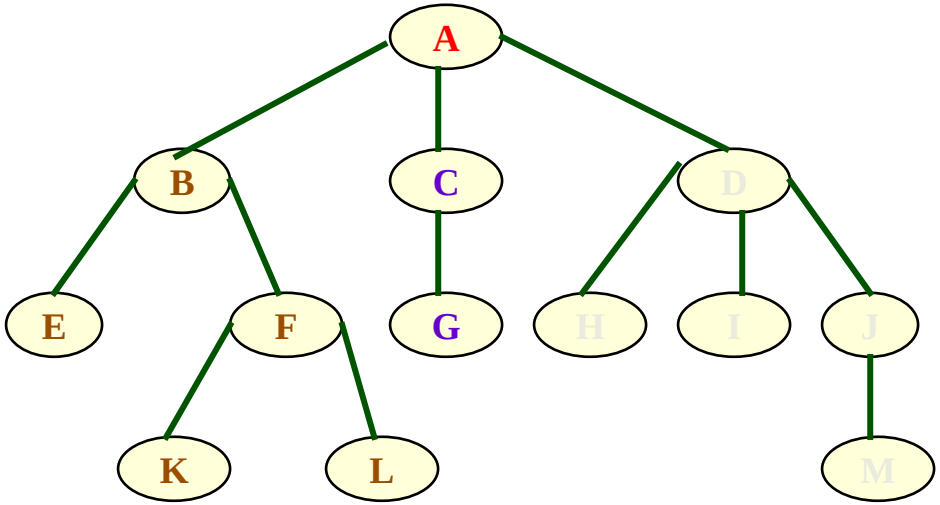
Basic Terms for Trees (6-3)

- Sibling: Nodes that have the same parent node are called siblings.
- Cousin: Nodes whose parents are at the same level are called cousins.
- Ancestor node: All nodes on the branches from the root to this node.
- Descendant node: any node in a subtree rooted at a node is called the descendant of that node;



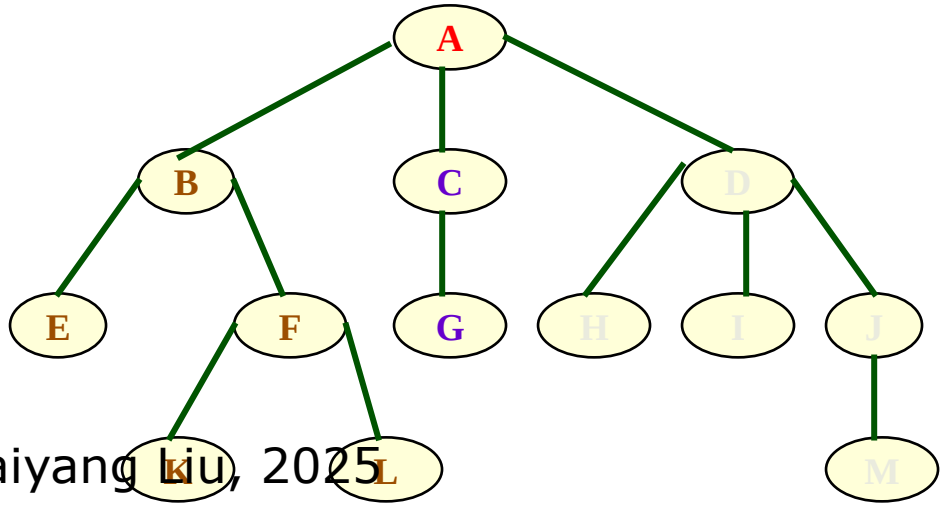
Basic terms for trees (6-4)

- Level of the node: the path length from the root node to that node plus 1;
- Tree depth: the maximum number of levels that a leaf node in a tree has;



Basic Terms for Trees (6-5)

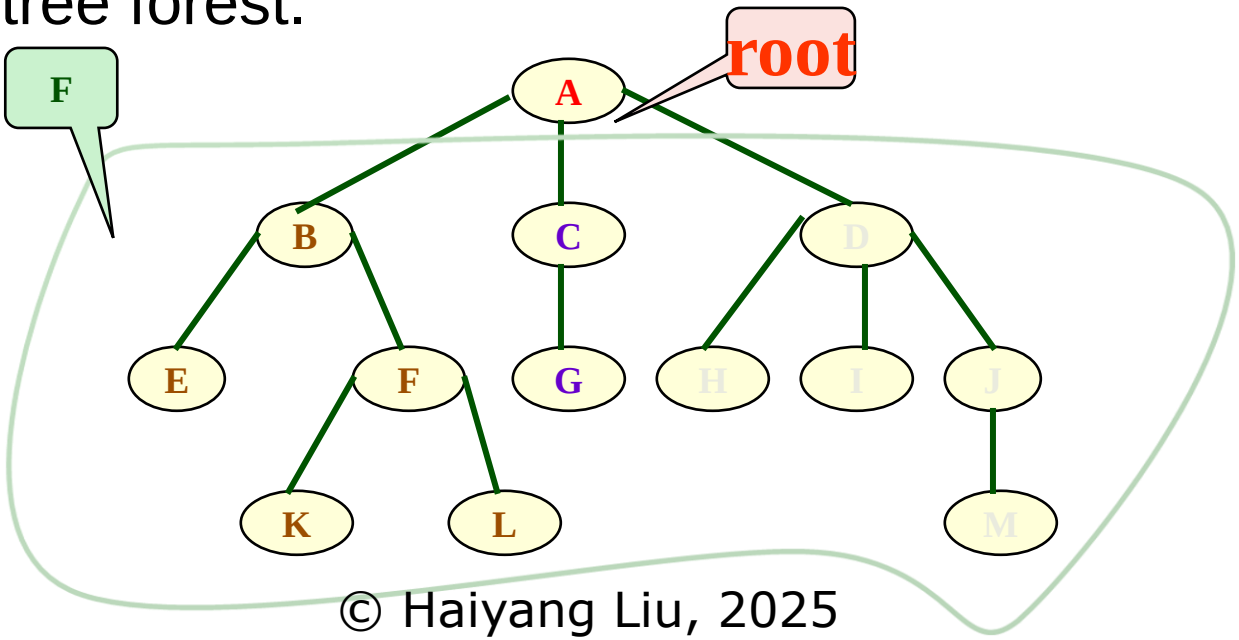
- Ordered tree: If the subtrees of each node in a tree are considered to be in a left-to-right order (i.e., they cannot be interchanged), then such a tree is called an ordered tree.
- First child: In an ordered tree, the root of the leftmost subtree is called the first child;
- Last child:The root of the rightmost subtree is called the last child.



© Haiyang Liu, 2025

Basic terms for trees (6-6)

- A forest is a set of m (m ≥0) disjoint trees. For each node in the tree, the set of its subtrees is called the forest.
- Any non-empty tree can be represented as a tuple Tree=(root, F), where root is the root node and F is called the subtree forest.



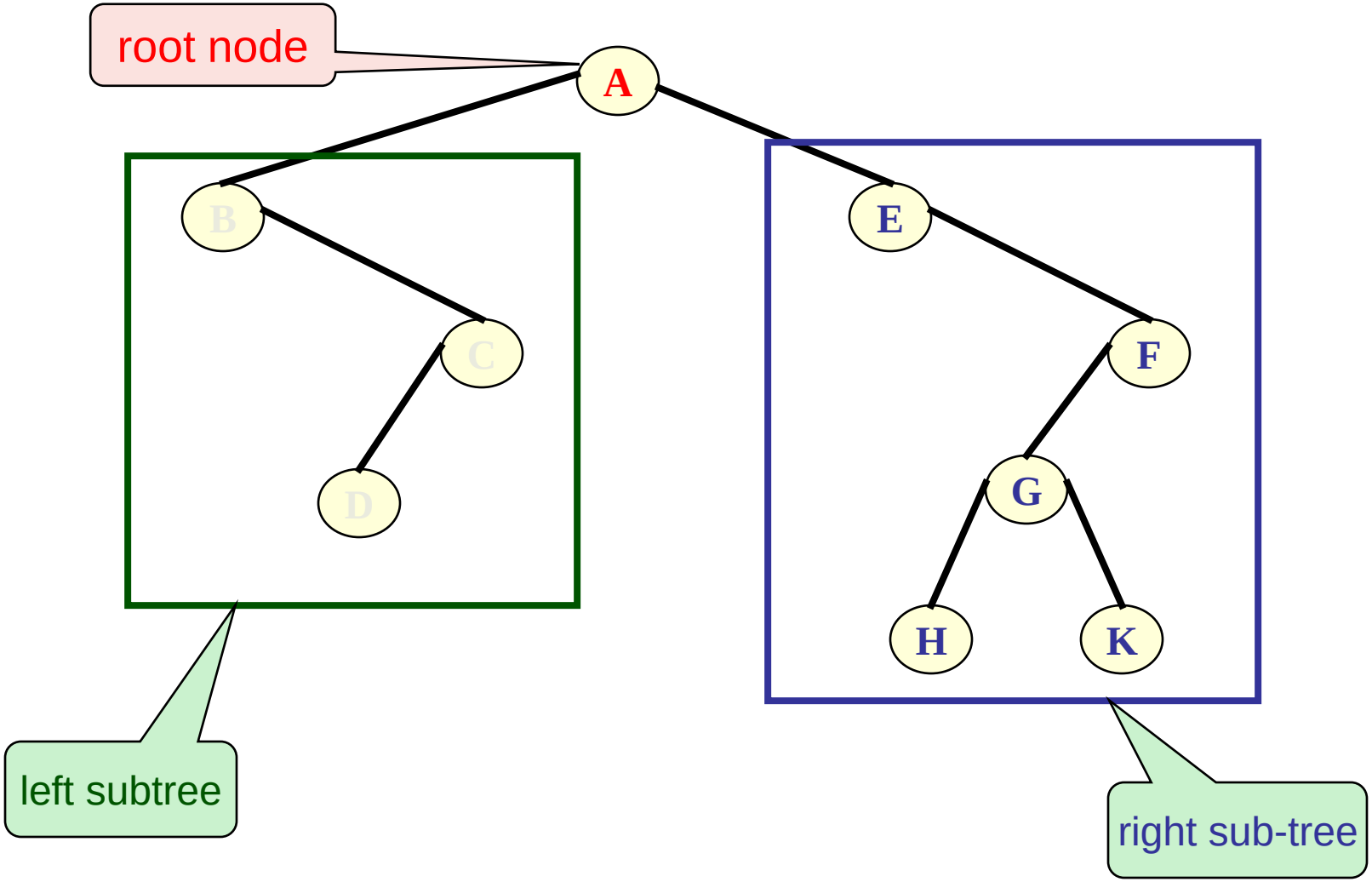
Main content

- Tree type definitions
- **Type definition of binary tree**
- Properties of a binary tree
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

Definition of binary tree

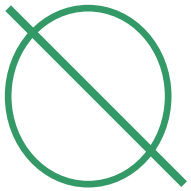
- A binary tree is a finite set of n (n≥0) nodes, which is either an empty set or consists of a root node and two disjointed left and right subtrees, which are themselves binary trees.
- Characteristic：
  - Each node has at most two subtrees,
  - The subtrees of a binary tree are distinguished as left and right.
  - Their order cannot be reversed arbitrarily, and they are disjoint.

An example of a binary tree

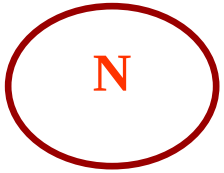


Five basic forms of a binary tree

empty tree

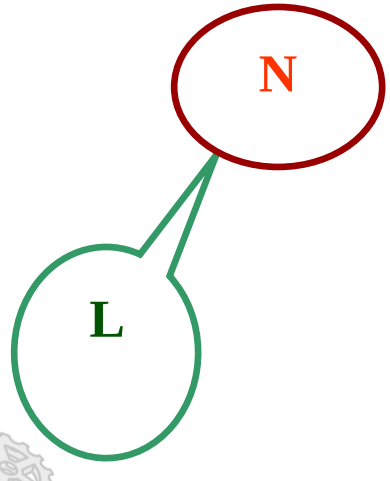


Only root node

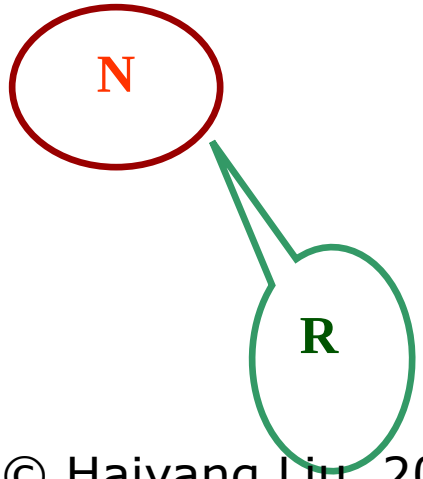


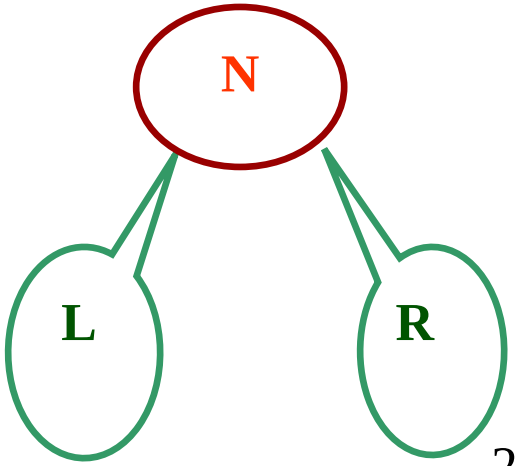
Neither left nor right subtree is empty

The right sub-tree is empty



The left sub-tree is empty





© Haiyang Liu, 2025

Basic operations of a binary tree

- Searching
- Insertion
- Deletion

Binary tree basic operations: searching (1)

- Root(T) // Find the root;
- Value (T, e); // Get the value of an element at the current node;
- Parent (T, e); // Returns the parent node of the current node;
- LeftChild (T, e) or RightChild (T, e);

//Find the child nodes;

- LeftSibling (T, e) or RightSibling (T, e); // Find the siblings;
- BiTreeEmpty (T); // Determines whether the tree is empty or not;
- BiTreeDepth (T); // Calculate the depth

Binary tree basic operations: searching (2)

- Pre-order traversal operation:

  PreOrderTraverse(T, Visit());

- In-order traversal operation:

  InOrderTraverse(T, Visit());

- Post-order traversal operation:

  PostOrderTraverse(T, Visit());

- Level-order traversal operation:

  LevelOrderTraverse(T, Visit());

Binary tree basic operations: insertion

- InitBiTree(&T); // Construct an empty binary tree;
- Assign (T, &e, value); // Node assignment
- CreateBiTree(&T, definition); //Create a binary tree;
- InsertChild(T, p, LR, c); //Inserts the left/right subtree of P.

Binary tree basic operations: deletion

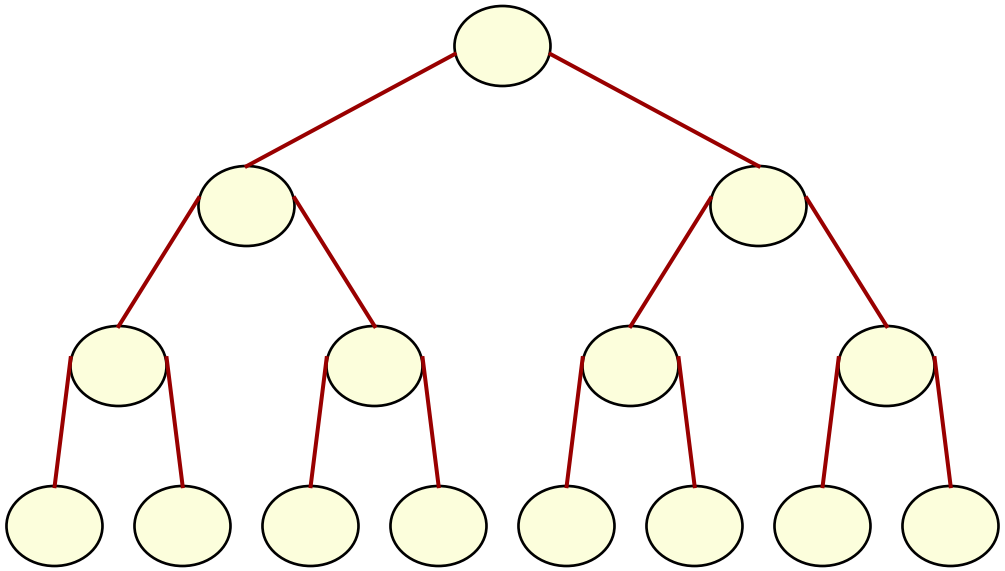
- ClearBiTree(&T); // Sets the binary tree to an empty tree.
- DestroyBiTree(&T); // Destroy binary tree
- DeleteChild(T, p, LR); // Delete subtree operation;

Main content

- Tree type definitions
- Type definition of binary tree
- **Properties of a binary tree**
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

Property 1

- The *i*-th level of a binary tree contains at most 2^{i-1} nodes (i ≥ 1).



Prove property 1 by induction

- 1. When $i = 1$, there is only one root node, which is obvious

$2^{i-1} = 2^0 = 1$ is correct;

- 2. Suppose that there are at most 2^{i-2} nodes on the i-1th layer. Since the degree of each node in the binary tree is at most 2, the maximum number of nodes on the i-th layer is twice the maximum number of nodes on the i-1th layer, that is, $2*2^{i-2} = 2^{i-1}$

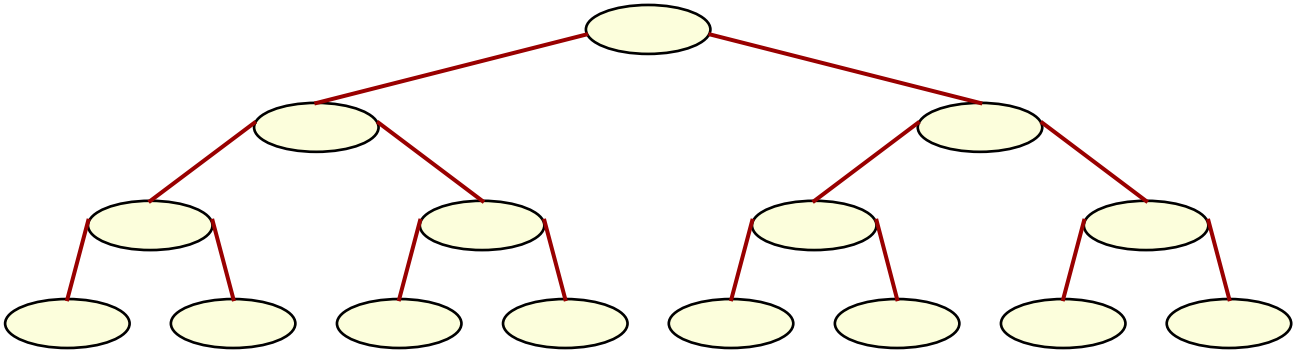
© Haiyang Liu, 2025

Property 2

- A binary tree with depth k contains at most 2^k-1 nodes ($k \geq 1$).
- Proof：

Based on the previous property, the number of nodes in a binary tree of depth k is at most:

$$2^0+2^1+\cdots\cdots+2^{k-1} = 2^k-1 \text{ 。}$$



Property 3

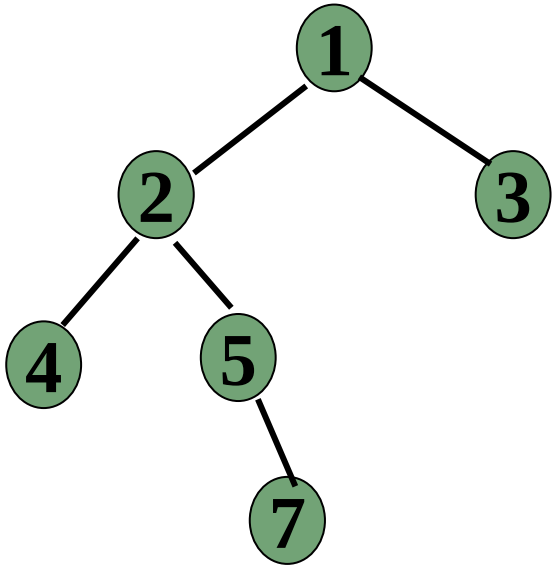
- For any binary tree that contains n_0 leaf nodes and n_2 nodes with degree 2, there must be a relation that:

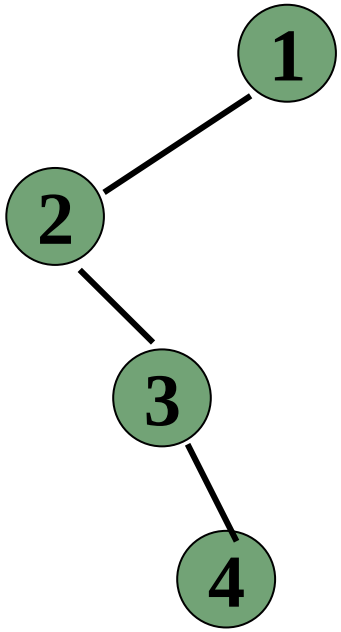
$$n_0 = n_2+1$$

- Proof：
- Let the total number of nodes in a binary tree be $n = n_0 + n_1 + n_2$
- The total number of branches on the binary tree is $b = n_1+2n_2$ (1)
- Expect the root node, all other nodes have a branch into them, so b = n-1.

  Substituting n into this equation gives us $b= n_0 + n_1 + n_2 – 1$ (2)
- Thus (1)-(2), we get $n_0 = n_2 + 1$.

Example of property 3

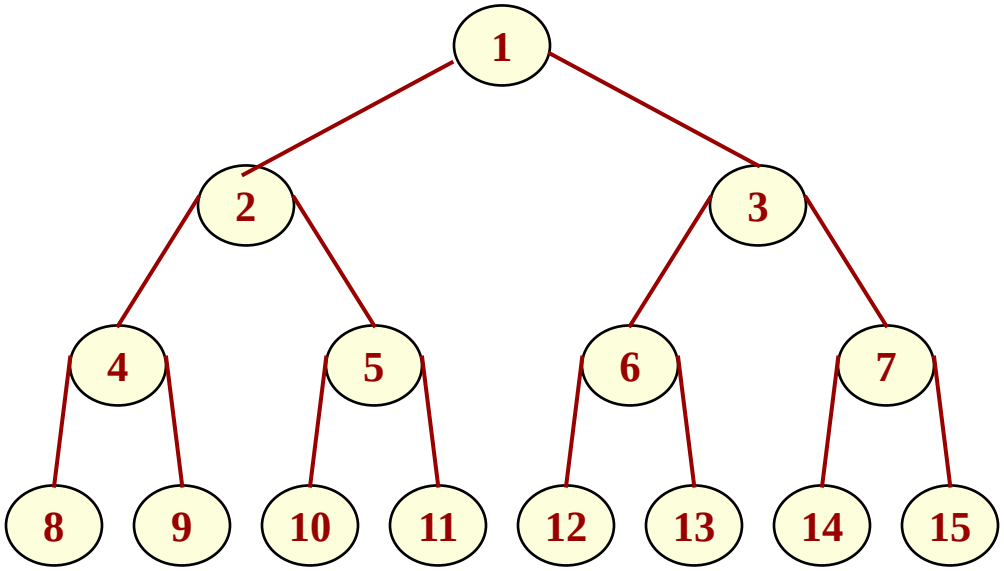




© Haiyang Liu, 2025

Two special types of binary trees

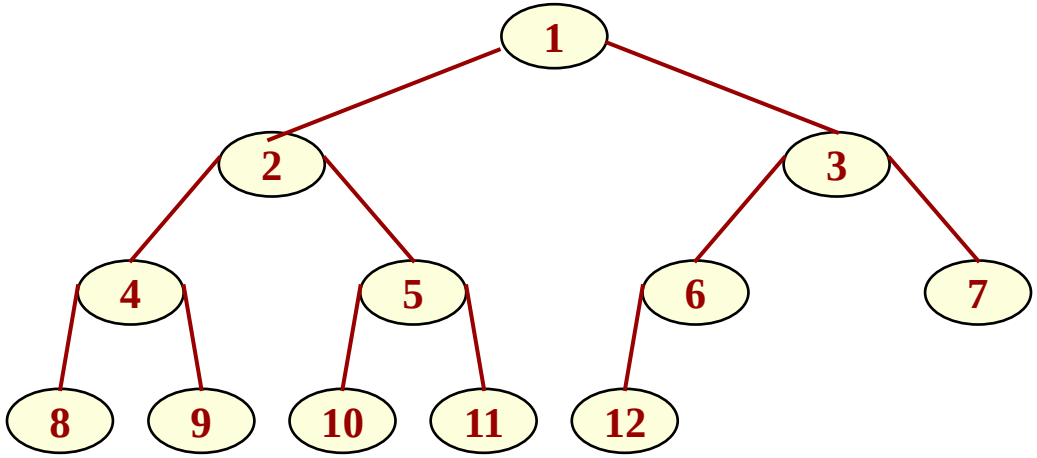
- Full binary tree: a binary tree of depth k with 2^k-1 nodes.
- Features: The number of nodes on each level is the maximum number of nodes.



© Haiyang Liu, 2025

Two special types of binary trees

- Complete binary tree: The *n* nodes contained in the tree correspond one-to-one to the nodes numbered *1 to n* in a full binary tree.
- Feature 1: Leaf nodes can only appear in the two layers with the largest levels.
- Feature 2: For any node, if the maximum level of its descendants under the right branch is L, then the maximum level of its descendants under the left branch must be L or L+1.



© Haiyang Liu, 2025

Property 4 (complete binary tree):

- The *depth* of a fully binary tree with *n* nodes is

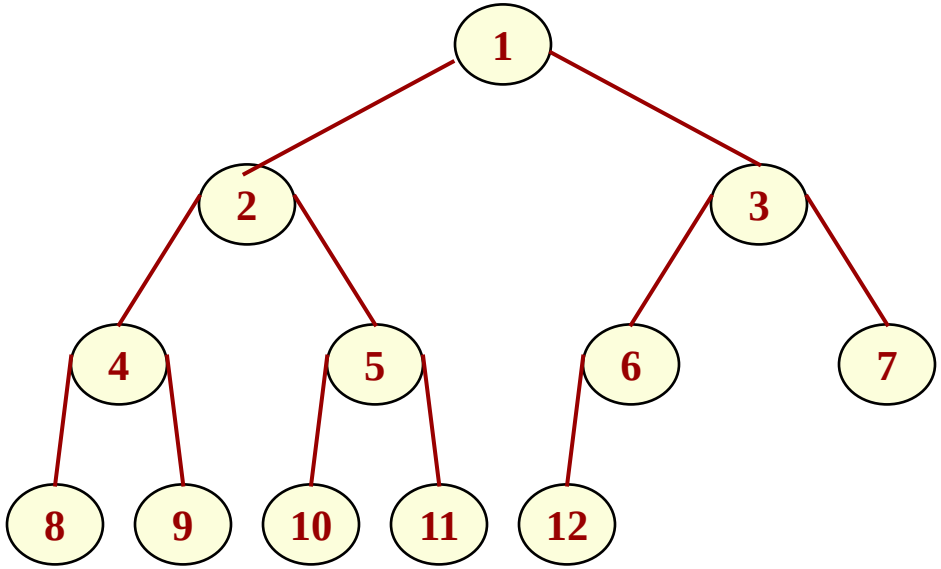
$$\lfloor log_2 n \rfloor + 1$$

- Proof:
- Let the depth of a complete binary tree be k
- Then according to the nature of Property 2,

  $2^{k-1} \leq n < 2^k$
- that $k-1 \leq log_2 n < k$
- Since k can only be an integer, $k = \lfloor log_2 n \rfloor + 1$

Example of property 4:

- The depth of a complete binary tree (a) is $\lfloor log_2 n \rfloor + 1 = \lfloor log_2 12 \rfloor + 1 = 3+1=4$
- The depth of a complete binary tree (b) is $\lfloor log_2 n \rfloor + 1 = \lfloor log_2 8 \rfloor + 1 = 3+1=4$ 。



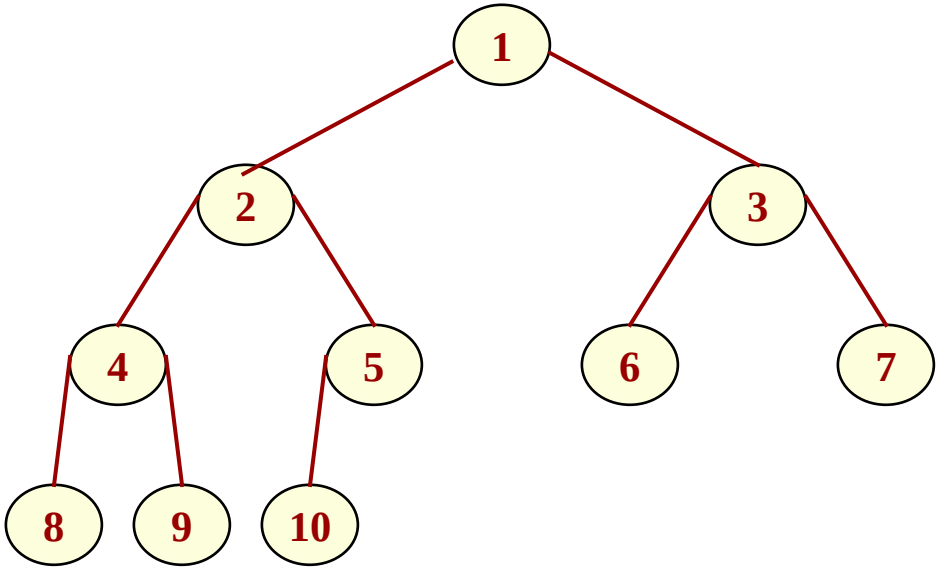
© Haiyang Liu, 2025

Property 5 (complete binary trees):

- If a complete binary tree with n nodes is numbered from 1 to n from top to bottom and left to right, then for any node numbered i in the complete binary tree:
  - If $i=1$, the node is the root of the binary tree and has no parent; otherwise, the node numbered $\lfloor i/2 \rfloor$ is its parent.
  - If $2i > n$, the node has no left child; otherwise, the node numbered $2i$ is its left child.
  - If $2i+1 > n$, the node has no right child; otherwise, the node numbered $2i+1$ is its right child.

Example of Property 5

- This complete binary tree contains 10 nodes, with node 1 serving as the root (which has no parent). The parent of node 5 is node 2 (i.e., $\lfloor 5/2 \rfloor = 2$). Since 2*6>10, all nodes with numbers greater than 6 have no left child. Node 4's left child is node 8 (2*4=8), and so on...



Main content

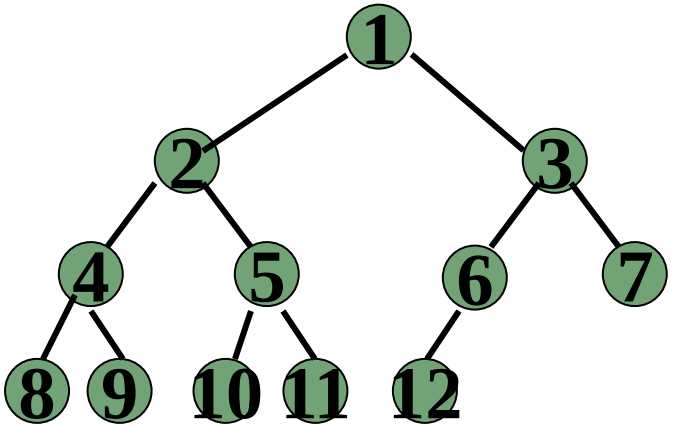
- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- **Storage structure of binary tree**
- binary tree traversal traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

Storage structure of binary tree

- Sequential storage representation of binary tree
- Chain storage representation of binary tree

Sequential storage representation of binary tree

- A set of consecutive memory units is used to store the node elements of a complete binary tree sequentially from top to bottom and left to right.

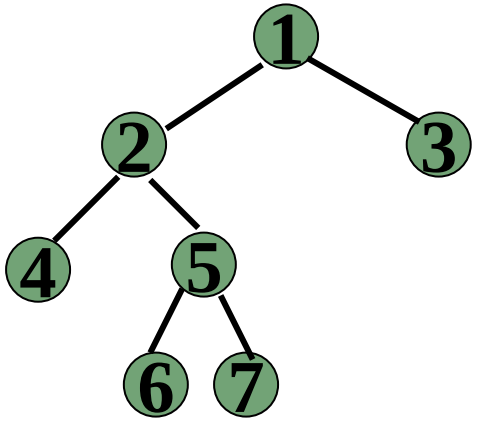


1	2	3	4	5	6	7	8	9	10	11	12

Sequential storage is suitable for complete binary trees

Sequential storage representation of binary tree

- For a general binary tree, each node should be compared to the nodes in a full binary tree and stored in the corresponding component of a one-dimensional array.



1	2	3	4	5	0	0	0	0	6	7

General binary tree:
(0 represents a non-existent node)

© Haiyang Liu, 2025

Description of C language for sequential storage

- #define MAX_TREE_SIZE 100

// The maximum number of nodes in a binary tree

- typedef TElemType SqBiTree[MAX_TREE_SIZE];

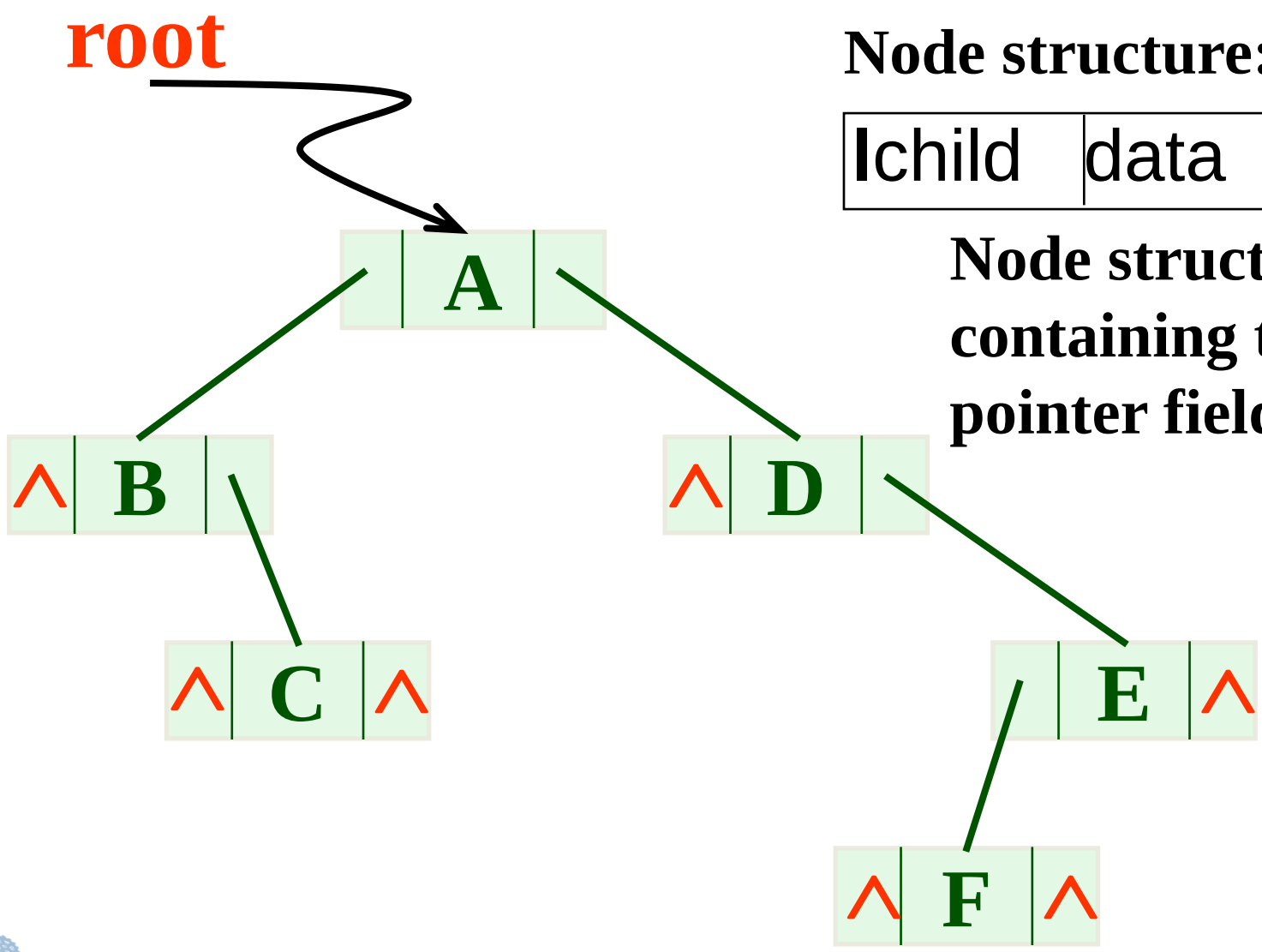
  // Unit 0 stores the root node

- SqBiTree bt;

Chain storage representation of binary tree

- Binary Linked List
- Ternary Linked List
- Parent Linked List
- Threaded Linked List

1.Binary linked list



Node structure:

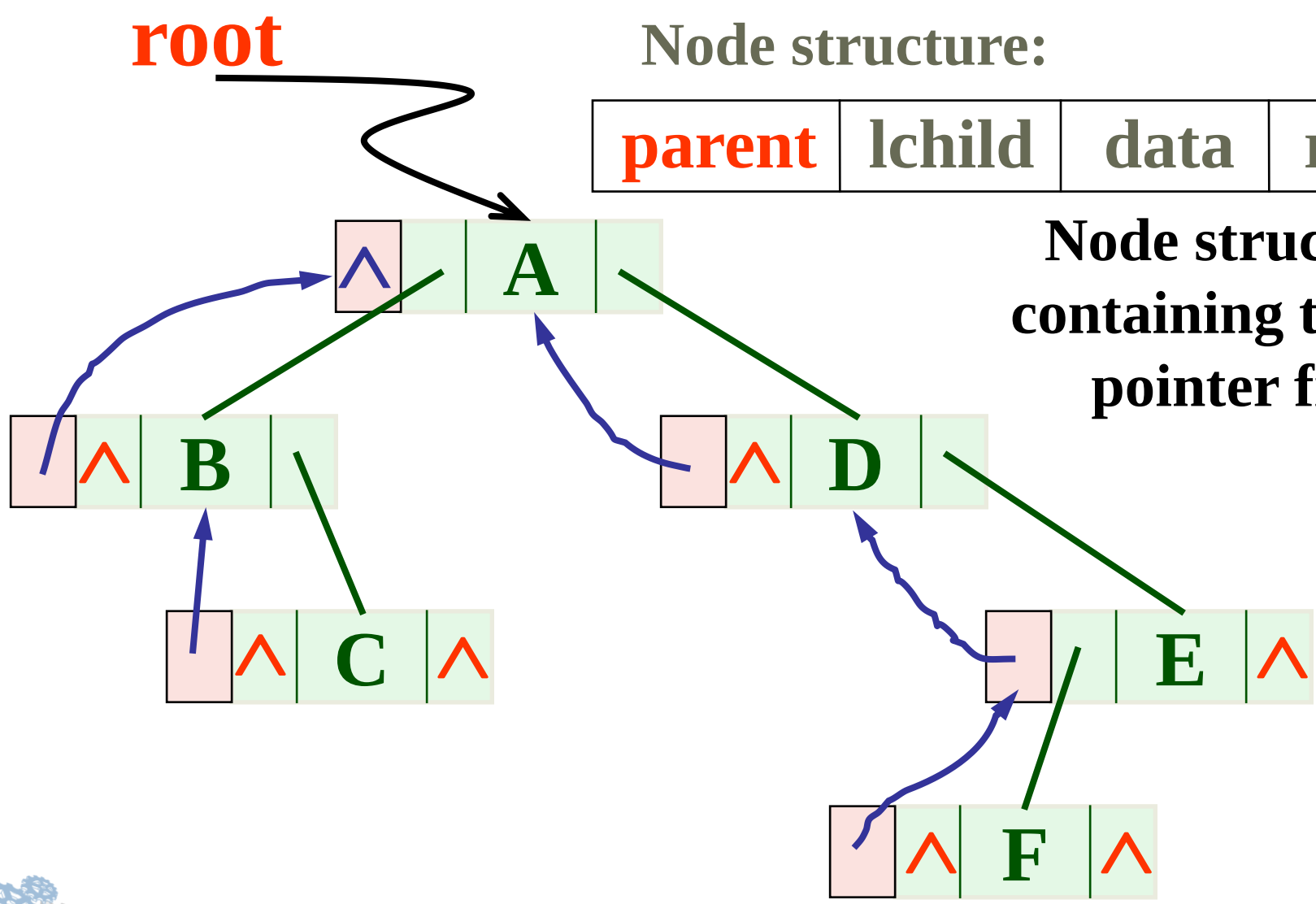
lchild	data	rchild

Node structure containing two pointer fields

C language description

```
typedef struct BiTNode { // Node structure
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
                        // Left and right child pointers
} BiTNode, *BiTree;
```

2. Ternary linked list



Node structure:

parent	lchild	data	rchild

Node structure containing three pointer fields

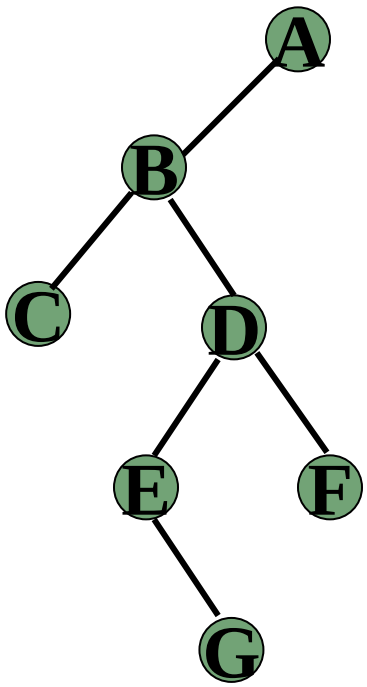
C language description

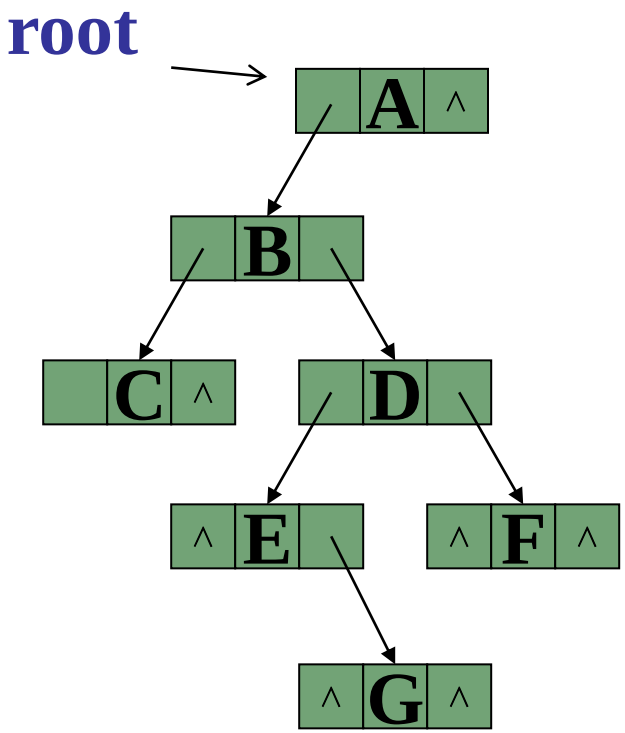
```
typedef struct TriTNode { // Node structure
    TElemType        data;
    struct TriTNode  *lchild, *rchild;
            // Left and right child pointers
    struct TriTNode  *parent;
            //Double parent pointer
} TriTNode, *TriTree;
```

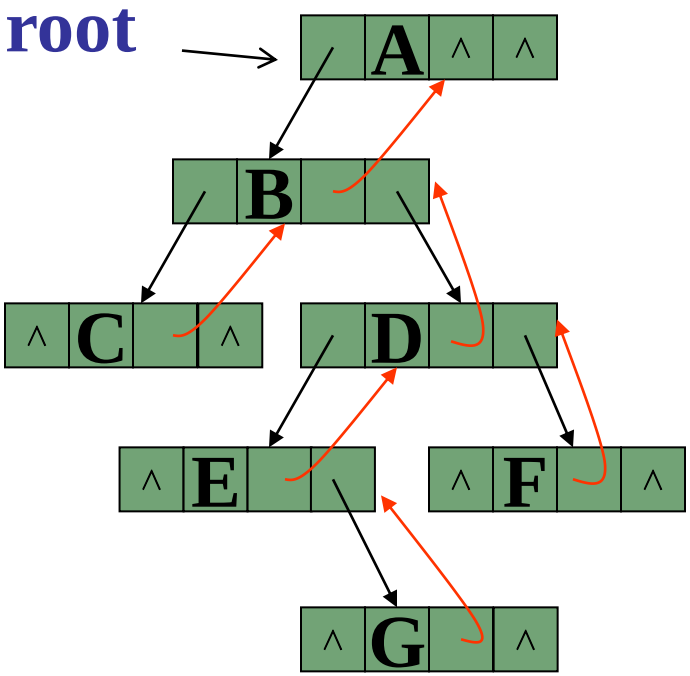
Node structure:

lchild	data	parent	rchild

example







Binary tree **Binary linked list** **Ternary linked list**

3. Parent linked list

0	B	2	**L**
1	C	0	**R**
2	A	-1	
3	D	2	**R**
4	E	3	**R**
5	F	4	**L**

Node structure:

data	parent	LRTag

© Haiyang Liu, 2025

C language description

```
typedef struct BPTNode {     // Node structure
        TElemType  data;
        int  parent;                    //  Pointer to the parent
        char  LRTag;      //Left and right child tag fields
    } BPTNode

    typedef struct BPTree{   // tree structure
        BPTNode nodes[MAX_TREE_SIZE];
        int  num_node;            // Number of nodes
        int  root;       // The location of the root node
    } BPTree
```

Main content

- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- **Binary tree traversal**
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

1. Binary tree traversal

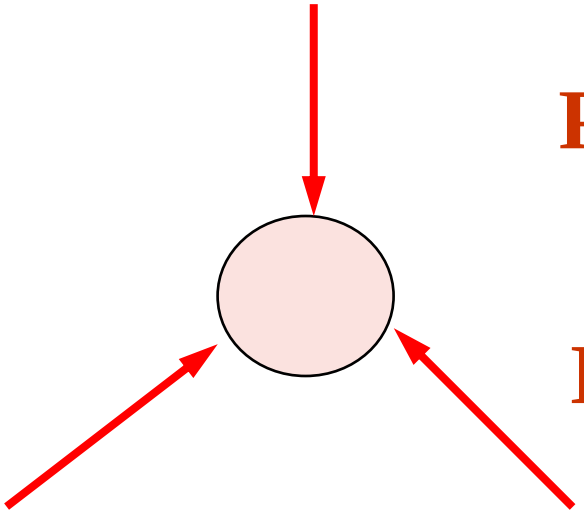
- Traverse the nodes in a binary tree along a search path so that each node is visited once and only once.
- The meaning of “visit" can be very broad, and various operations can be performed on the node, such as outputting the node information.
- "Traversing" is an operation applicable to all data structures. In linear structures, there exists a single search path (since each node has only one successor), making this concept unnecessary for discussion. However, in binary trees—a non-linear structure—each node has two successors, which raises the question of determining the search path for traversal.

Three search paths

For a binary tree, there are three search paths:

- 1. Level-order traversal from top to bottom;
- 2.Traversal that visits the left subtree first and then the right subtree;
- 3.Traversal that visits the right subtree first and then the left subtree.

2.Traversal that visits the left subtree first and then the right subtree;



Pre-order traversal algorithm

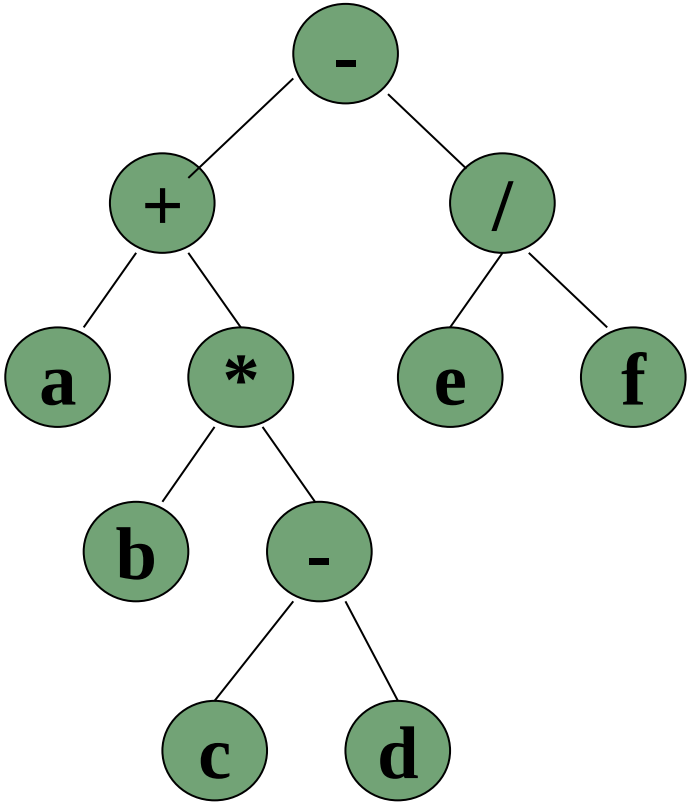
In-order traversal algorithm

Post-order traversal algorithm

© Haiyang Liu, 2025

Pre-order traversal algorithm

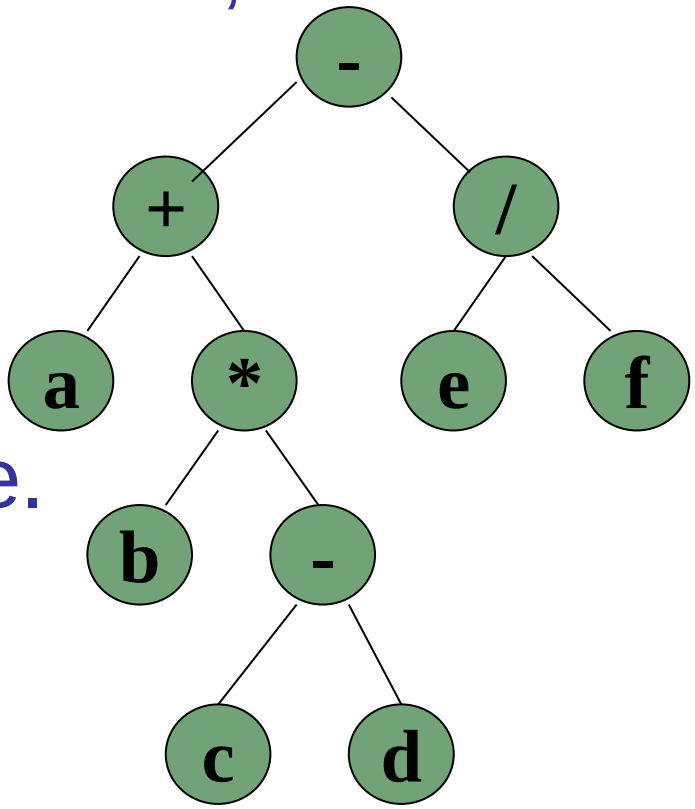
- If the binary tree is empty, the empty operation is performed; otherwise,
- (1) Visit the root node;
- (2) Preorder the left subtree;
- (3) Preorder the right subtree.



© Haiyang Liu, 2025

In-order traversal algorithm

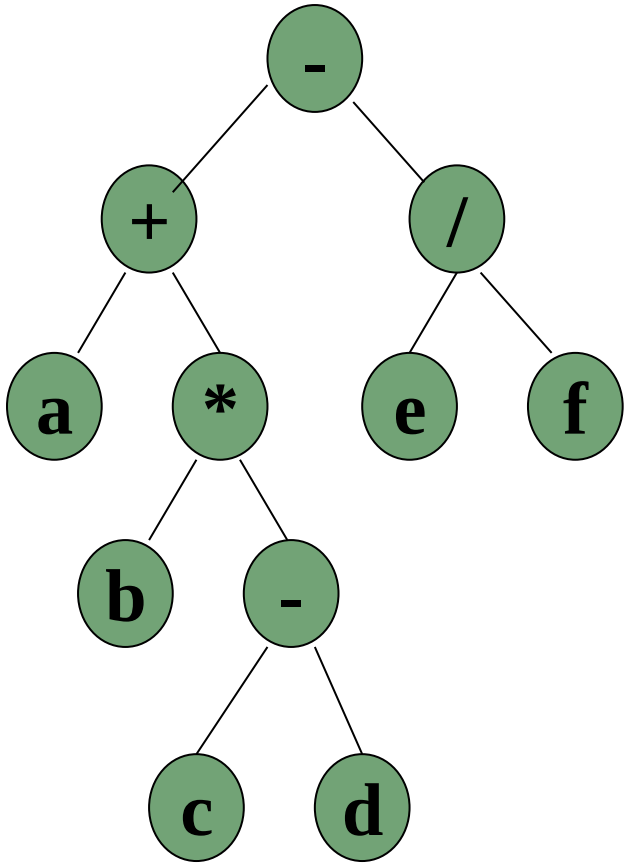
- If the binary tree is empty, the empty operation is performed; otherwise,
- (1) In-order the left subtree;
- (2) Visit the root node;
- (3) In-order the right subtree.



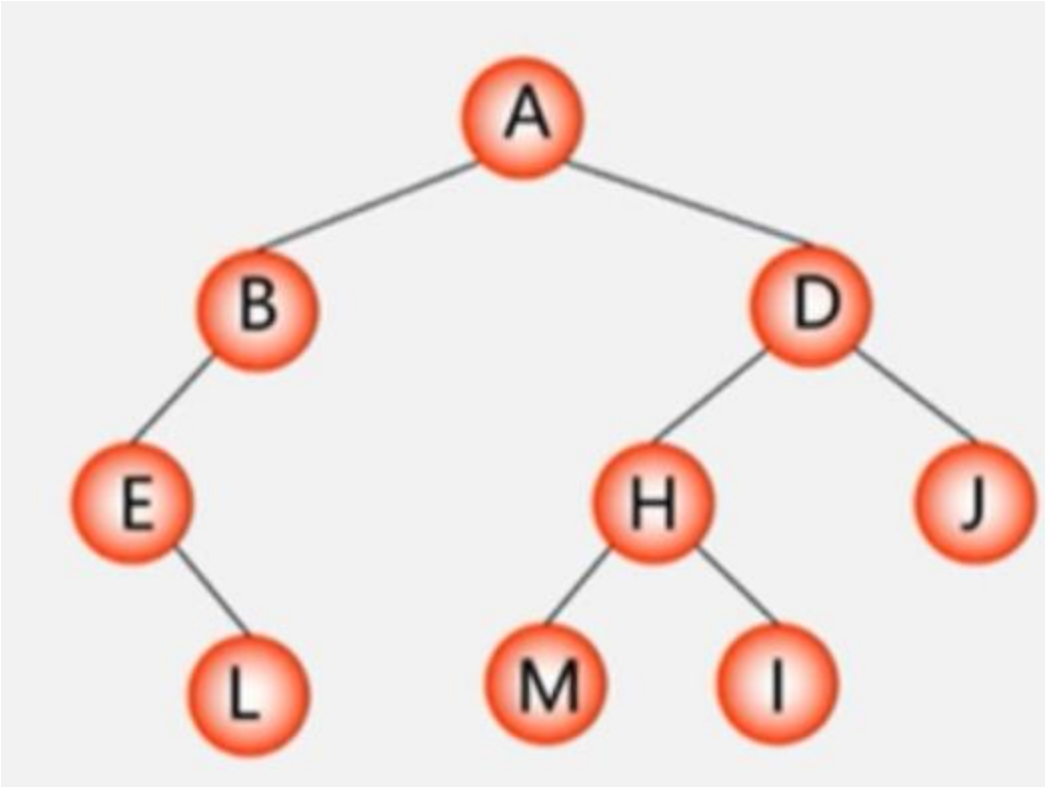
© Haiyang Liu, 2025

Post -order traversal algorithm

- If the binary tree is empty, the empty operation is performed; otherwise,
- (1) Post traverse the left subtree;
- (2) Post traverse the right subtree ;
- (3) Visit the root node.



Exercise



3.Recursive description of the algorithm (3-1)

- Pre-order traversal

```
void Preorder (BiTree T, void( *visit)(TElemType& e))
{                    // Preorder a binary tree
   if (T) {
      visit(T->data);                    // Visit the root node
      Preorder(T->lchild, visit); // Traverse left subtree
      Preorder(T->rchild, visit);// Traverse right subtree
   }
}
```

3.Recursive description of the algorithm (3-2)

- In-order traversal

```
void Inorder (BiTree T, void( *visit)(TElemType& e))
{                                   // In-order traversal binary tree
  if (T) {
    Inorder(T->lchild, visit);  // Traverse left subtree
    visit(T->data);                 // Visit the root node
    Inorder(T->rchild, visit);  //  Traverse right subtree
  }
}
```

3.Recursive description of the algorithm (3-3)

- Post-order traversal

```
void Postorder (BiTree T, void( *visit)(TElemType& e))
{          // Postorder traversal of a binary tree
  if (T) {
    Postorder(T->lchild, visit);  //  Traverse left subtree
    Postorder(T->rchild, visit); // Traverse right subtree
    visit(T->data);                    // Visit the root node
  }
}
```

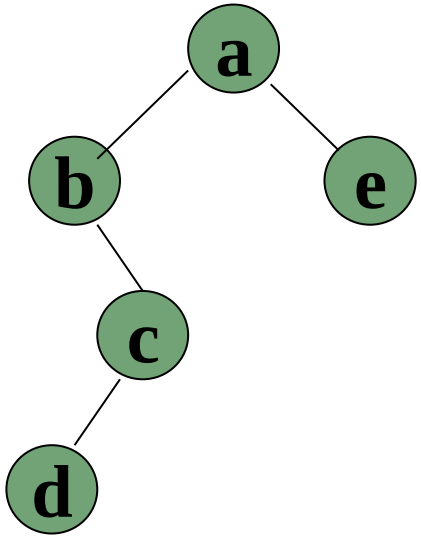
The recursive algorithm is concise and refined,

but the efficiency is low.

© Haiyang Liu, 2025

4.Non recursive algorithm for traversing a binary tree

- In a non-recursive algorithm, a stack is used to store the left and right children of the nodes encountered during traversal.



- The basic idea of using stack to implement in-order traversal of binary tree is as follows:
- **Starting from the root node of the binary tree, walk along the left subtree to the end. During the forward search, the nodes encountered are pushed into the stack. When the left subtree is traversed, the nodes are popped from the top of the stack and accessed, and then the right subtree is traversed.**

Non-recursive description of the in-order traversal algorithm (1-1)

```
BiTNode *GoFarLeft(BiTree T, Stack *S)
 {
     if (!T )  return NULL;
       while (T->lchild ) {  // Locate the bottom-left node
         Push(S, T);
         T = T->lchild;
       }
     return T;
}
```

Non-recursive description of the in-order traversal algorithm (1-2)

```
void Inorder_I(BiTree T, void (*visit)(TelemType& e))
//  Non-recursive algorithm for in-order traversing a binary tree
{  Stack *S;
    t = GoFarLeft(T, S);              // Locate the bottom-left node
   while(t)
   { visit(t->data);
      if (t->rchild)
         t = GoFarLeft(t->rchild, S);
     else if ( !StackEmpty(S ))    //  Pop the stack if its not empty
        t = Pop(S);
             else    t = NULL;        //  This indicates the end of the traversal.
    } // while
}// Inorder_I
```

Non-recursive description of the in-order traversal algorithm (2-1)

- Idea:

Consider a binary tree stored in a linked list. A sequential stack is used to store returned nodes. The process begins by scanning all left children of the root node and pushing them onto the stack. Each node is then popped and examined, followed by scanning its right children and pushing them onto the stack. This sequence continues until the stack becomes empty.

Non-recursive description of the mid-traversal algorithm (2-2)

```
void inorder( BiTree  b)
  // Non-recursive algorithm for mid-traversal binary tree 2
  { BiTree *stack[m0],*p;
     int top=0;
     p=b;
     do
       { while (p!=NULL)            //Scan all left nodes
           { top++;
              stack[top]=p;
              p=p->lchild;
           }
```

Non-recursive description of the mid-traversal algorithm (2-3)

```
        if (top>0)
          { p=stack[top];
            // p refers to a node without a left subtree
           // Or its left subtree has been traversed
            top--;
            printf("%d",p->data); //Access node
            p=p->rchild;              //Scan the right subtree
          }
      } while (p!=NULL || top!=0)
  }
```

Non-recursive algorithm of pre-order traversal (1)

- Assume that the binary tree is stored in a binary linked list, and use a stack to implement a nonrecursive pre-order traversal.

- Algorithm idea:

First analyze the characteristics of the output order of nodes in the pre-order traversal (the root is output first, and the right child is output later)

Non-recursive algorithm for pre-traversal (2)

```
void preorder( BiTree  b)
 // A non-recursive algorithm for pre-traversing a binary tree
   { BiTree *stack[m0];
     int top;
     if (b!=NULL)
       { top=1;                          //root node into stack
          stack[top]=b;
          while (top>0)                 //The loop is not empty
             { p=stack[top];    //pop and access the node
                top--;
                printf("%d",p->data);
```

Non-recursive algorithm for pre-traversal (3)

- Which of the left and right children goes into the stack first?

```
        if (p->rchild!=NULL)     // right child into stack
            { top++;
              stack[top]=p->rchild;}
          if (p->lchild!=NULL)    //left child into stack
            { top++;
              stack[top]=p->lchild;
            }
        }//while
    }//if
}
```

Non-recursive algorithm for post-traversal (1)

Algorithm idea:

- 1.Use a stack to save the returned nodes;
- 2.First, scan all the left nodes of the root node and put them into the stack. Then pop a node out of the stack, scan the right nodes of the node and put them into the stack, and then scan all the left nodes of the right node and put them into the stack;
- 3.When a node is visited after both left and right subtrees have been visited, the node is visited again until the stack is empty.

Non-recursive algorithm for post-traversal (2)

- If returning from the left subtree to the root, the next step is to continue traversing the right subtree; if returning from the right subtree to the root, the root node should be visited. This raises a question: How do we distinguish between returning from the left subtree and the right subtree when popping back to the root?

- This algorithm employs two stacks (stack1 and stack2) and a shared top pointer that stores two values: one indicating the pointers value and another marking the subtree type (0 for left subtree, 1 for right subtree). During stack popping, the system distinguishes between left and right subtree returns by checking the node pointers value, which determines whether to proceed to the right subtree or return to the root node.

Non-recursive algorithm for post-traversal (3)

```
void postorder( BiTree  b)
    // Non-recursive algorithm for post-traversal of binary tree
{ BiTree *stack[m0],*p;
  int tag[m0], top=0;   p=b;
do
  { while (p!=NULL)    //Scan left nodes
      { top++;   stack[top]=p; tag[top]=0;
        p=p->lchild; }
     while ((top>0) && tag[top]==1)
     // Both left and right subtrees of p have been visited
```

Non-recursive algorithm for post-traversal (4)

```
        while ((top>0) && tag[top]==1)
         //  Both left and right subtrees of p have been visited
         { printf("%d", stack[top] ->data);
            top--; }//Access the node and pop the stack
        p=stack[top];
       if ((top>0) && (tag[top]==0))
         //Scan the right subtree
         { p=p->rchild; tag[top]=1; }
   } while (p!=NULL || top!=0)
}
```

Algorithm to traverse a binary tree in order 1

- An algorithm that traverses a binary tree in order (the same level from left to right)
- Algorithmic idea **(width-first algorithm)**:

Assuming the binary tree adopts a binary linked list structure, this algorithm utilizes a queue q. The process begins by inserting the root node into the queue, then dequeuing and outputting it. If the node has a left subtree, its root is added to the queue; if it has a right subtree, the right subtrees root is inserted. This sequence continues until the queue becomes empty. By leveraging the first-in-first-out property of queues, the algorithm achieves hierarchical traversal of the binary tree.

Algorithm to traverse a binary tree in order 2

```
#define MAXLEN 100
void translevel(BiTree b)          //Traverse by level
 {  struct node
      { BiTree *vec[MAXLEN];
        int f,r;
      } q;
    q.f=0;
    q.r=0;                         //Put the queue empty queue

     if (b!=NULL) printf("%d",b->data);
       q.vec[q.r]=b;   //Node pointer enters the queue
       q.r=q.r+1;
```

Algorithm to traverse a binary tree in order 3

```
while (q.f<q.r)                // the queue is not empty
  { b=q.vec[q.f];   q.f=q.f+1;   // dequeue
    if (b->lchild!=NULL)       //Output the left child and add it to the queue
      {  printf("%d",b->lchild->data);
         q.vec[q.r]=b->lchild;
         q.r=q.r+1;
      }
  if  (b->rchild!=NULL)      //Output the right child and add it to the queue
       { printf("%d",b->rchild->data);
         q.vec[q.r]=b->rchild;
         q.r=q.r+1;
       }
    }
  }
```

Binary tree traversal traversal of binary tree

- Problem Statement
- A traversal algorithm that goes left, right, and then back
- A recursive description of the algorithm
- A non-recursive description of the traversal algorithm
- Examples of the application of traversal algorithm

Examples of the application of traversal algorithm

1. Count the number of leaf in a statistical binary tree (preorder)
2. Find the depth of the binary tree (postorder)
3. Find the ancestors of a binary tree with value X (post-order)
4. Copy the binary tree (post-order)

Self study

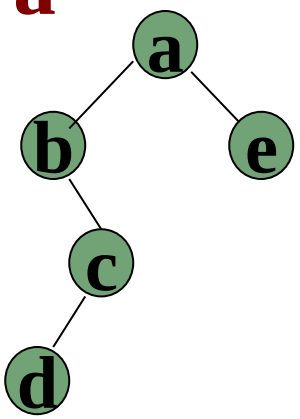
5. Establish the storage structure of the binary tree
6. Construct a binary tree based on the traversal sequence

© Haiyang Liu, 2025

The number of leaf nodes in a statistical binary tree

- Preorder (or mid-order or post-order) traverse a binary tree, looking for leaf nodes and counting them as you go.
- Therefore, it is necessary to add a "count" parameter to the traversal algorithm and change the "access node" operation in the algorithm to: if it is a leaf, the counter increases by 1.

1. Count the number of leaf nodes in a statistical binary tree



Basic idea of algorithm:

Preorder (or mid-order or post-order) traverse a binary tree, looking for leaf nodes and counting them as you go. Therefore, it is necessary to add a "count" parameter to the traversal algorithm, and change the operation of "accessing nodes" in the algorithm to: if it is a leaf, the counter is increased by 1.

© Haiyang Liu, 2025

```
void CountLeaf (BiTree T,  int& count)
// Count the number of leaf nodes in a binary tree
{   if ( T )
    {  if ((!T->lchild)&& (!T->rchild))
           count++;     // Counting leaf nodes

       CountLeaf( T->lchild, count);
       CountLeaf( T->rchild, count);

    } // if
} // CountLeaf
```

Method two for counting the number of leaves in a statistical binary tree

```
Int CountLeaf (BiTree T)
//Method two for counting the number of leaves in a binary tree
{ if (T==NULL)  return 0;
   else if (T->lchild==NULL &&
                T->rchild==NULL)  return 1;
   else  return
     CountLeaf(T->lchild) +CountLeaf(T->rchild);
}
```

2. Find the depth of the binary tree (postorder traversal)

- **Basic idea of algorithm:**

First analyze the relationship between the depth of a binary tree and the depth of its left and right subtrees.

- **According to the definition of binary tree depth, the depth of a binary tree should be the maximum depth of its left and right subtrees plus 1. Therefore, we need to calculate the depths of the left and right subtrees separately. The "node access" operation in the algorithm is: calculate the maximum depth of the left and right subtrees, then add 1.**

```
int Depth (BiTree T ){   // Return the depth of a binary tree
  if ( !T )    depthval = 0;
  else {
     depthLeft = Depth( T->lchild );
     depthRight= Depth( T->rchild );
     depthval = 1 + (depthLeft > depthRight  ?
                                  depthLeft : depthRight);
  }
  return depthval;
}
```

5. Establish a binary tree storage structure

There are many algorithms to build a binary tree storage structure

(1) In string format: root-left subtree-right subtree (Pre-order form) Define a binary tree

(2) Construct a corresponding binary tree based on the expression

(3) Construct a binary tree from its preorder and inorder sequences

© Haiyang Liu, 2025

(1) In string format: **root-left subtree-right subtree** **(Preceding form)** Define a binary tree

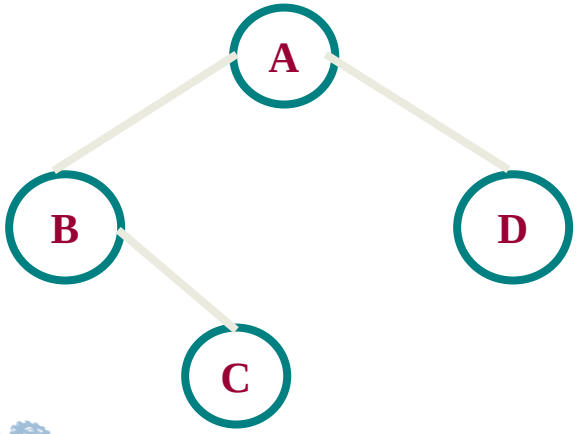
for instance：

empty tree

Represented by the blank character "■"

A binary tree with only one root node

represented by the string "A(■ ■)"



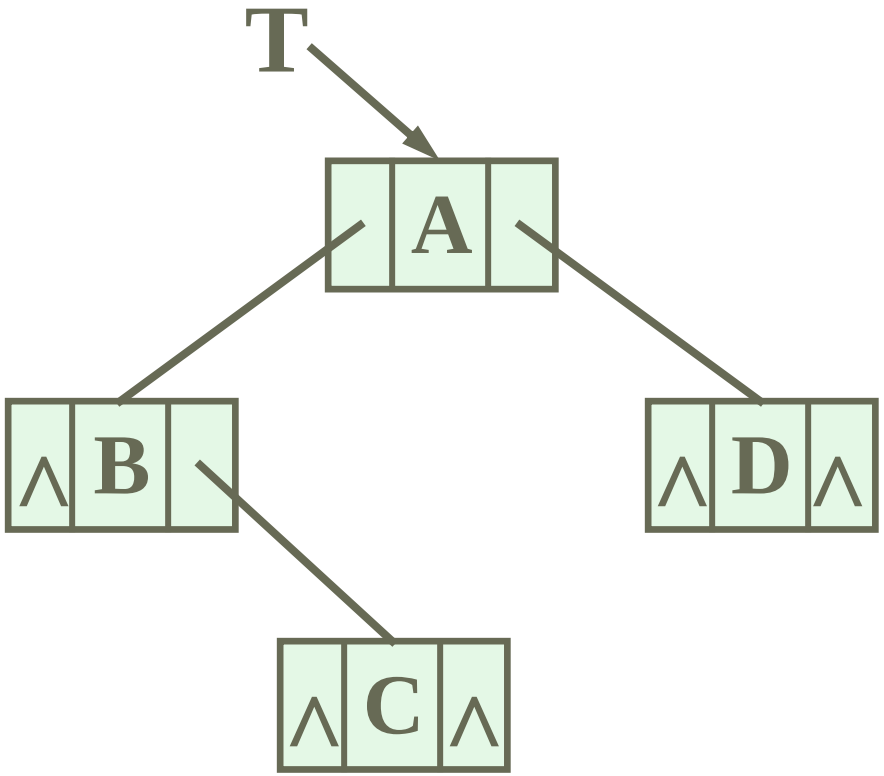
Represents the following string

A(B(■,C(■,■)),D(■,■))

```
Status CreateBiTree(BiTree &T)
//Enter the values of the binary tree nodes in preorder
 {  scanf(&ch);
    if (ch==' ')  T = NULL;
    else {
      if (!(T = (BiTNode *)malloc(sizeof(BiTNode))))
              exit(OVERFLOW);
     T->data = ch;                          // Generate root node
     CreateBiTree(T->lchild);     // Construct left subtree
     CreateBiTree(T->rchild); } //Construct the right subtree
    return OK;
 } // CreateBiTree
```

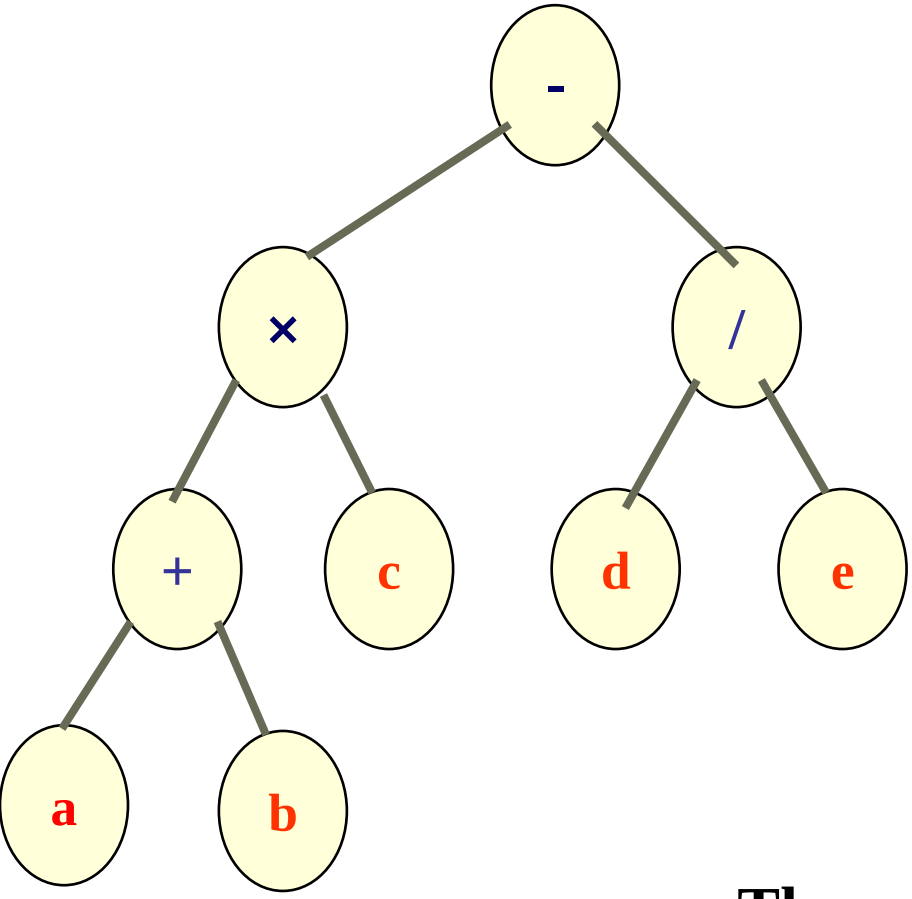
The execution process of the algorithm on the previous page is as follows:

A B ■ C ■ ■ D ■ ■



(2) Build the corresponding binary tree according to the expression

It is known that the expression (a+b)× c-d/e



Preorder traversal: - ×+ abc /de

--- prefix expression

Intermediate traversal: a +b×c-d/e

--- nifix expression

Backtracking traversal: a b +c×de / -

---Postfix expressions

characteristic：

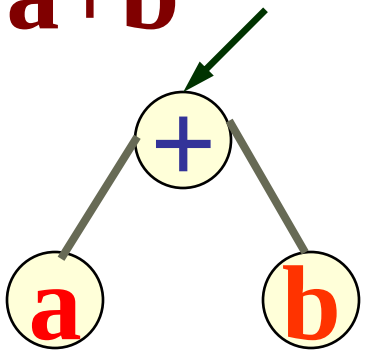
The operand is a **leaf node**

The operator is a **branch node**

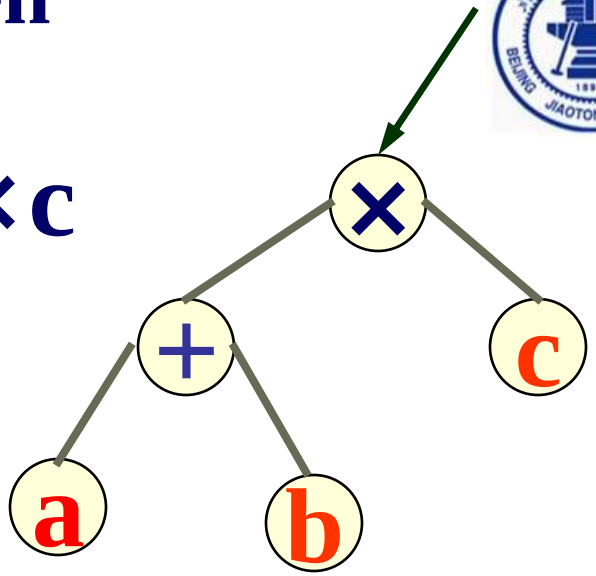
© Haiyang Liu, 2025

Analyze the relationship between expressions and binary trees:

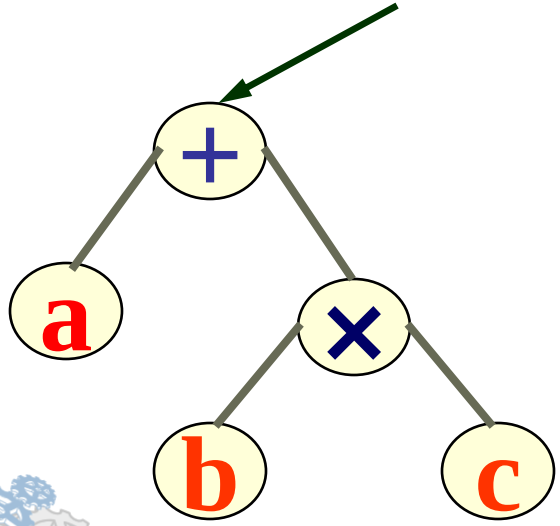
a+b



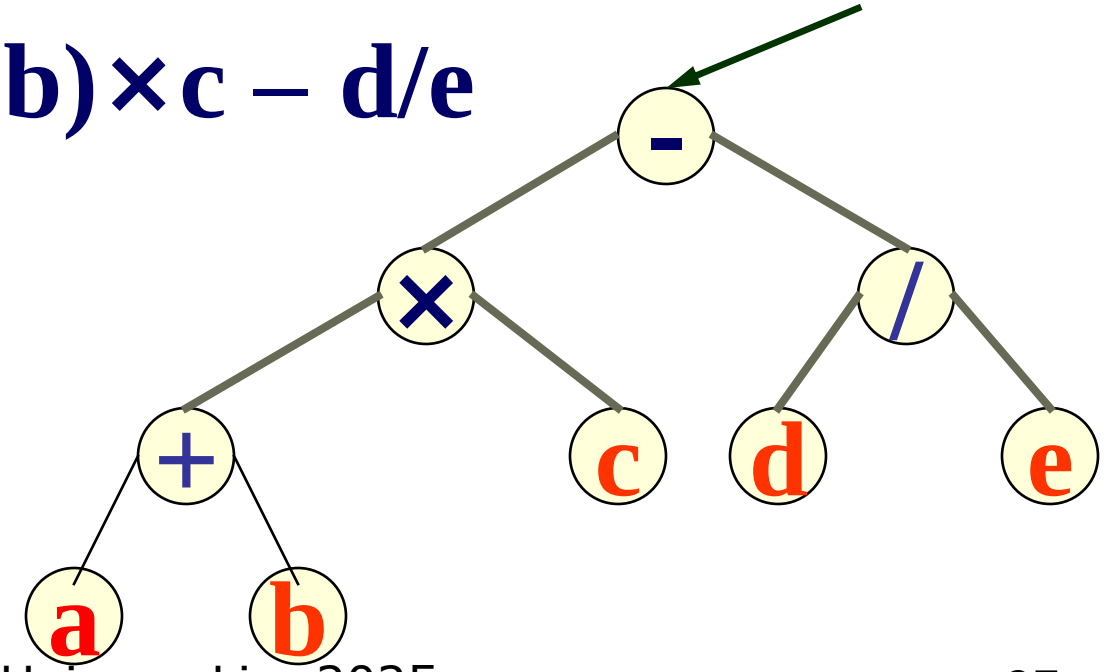
(a+b)×c



a+b×c



(a+b)×c – d/e



© Haiyang Liu, 2025

Algorithmic thinking

Use two stacks: one for storing leaves or subtrees, and one for storing operators

- Read a character;
- If it is a number, build a leaf node and enter the tree stack temporarily;
- If the operator is a character, compare the priority with the top element of the character stack.
  - If the current priority is "high", the operator is temporarily stored on the stack.
  - If the top of the stack has a high priority, the operator stacks top element (root) is popped, and two elements (right and left children) are removed from the tree stack to form a new subtree. This new subtree is then pushed back into the tree stack for storage.

The algorithm for creating leaf nodes is:

```
void CrtNode(BiTree& T,char ch)
{
  T=(BiTNode*)malloc(sizeof(BiTNode));
  T->data = ch;
  T->lchild = T->rchild = NULL;
  Push( PTR, T );
 //Insert leaf node into PTR tree stack
}
```

The algorithm for constructing a subtree is:

```
void CrtSubtree (Bitree& T, char c)
{
  T=(BiTNode*)malloc(sizeof(BiTNode));
  T->data = c;
  Pop(PTR, rc);  T->rchild = rc;  //Pop two leaf nodes
  Pop(PTR, lc);  T->lchild = lc;  //construct subtree
// Traverse left-to-right, pushing left child nodes into the stack first.
// Therefore, the right subtree must be popped first
  Push(PTR, T);     //Insert the subtree into the PTR stack
}
```

© Haiyang Liu, 2025

```
void CrtExptree(BiTree &T, char exp[] )
{ InitStack(S);  Push(S, '#');  InitStack(PTR);
  p = exp;  ch = *p; // PTR stack stores trees and nodes, S stack
stores characters and operators
  while (!(GetTop(S)=='#' && ch=='#')) {
    if (!IN(ch, OP))   CrtNode( t, ch );//Character read,
 // Create a leaf node and push it onto the stack. #: marks the start
//and end of an expression. OP: represents the set of operators.
    else { . . .  . . .}          // indicates that the operator is read
    if ( ch!= '#' ) { p++;  ch = *p;}//Advance pointer if not reached end
    } // while
  Pop(PTR, T);     //Pop the last tree from the PTR stack
} // CrtExptree
```

© Haiyang Liu, 2025

```
switch (ch) {
   case '(' : Push(S, ch); break; //indicates that the read ch is "("
   case ')' : Pop(S, c);          // indicates that the read ch is ")"
              while (c!= '(' ) {
              //The stack top is not an opening parenthesis
                  CrtSubtree( t, c);  //Construct a subtree based on the
                  //stacks top element and push it onto the stack
                  Pop(S, c) ; }
               break;
   defult :  … …
 //indicates that the read character ch is another operator

} // switch
```

```
while(!Gettop(S, c) && ( precede(c,ch))) {
    //The operation priority of the stacks top element c //is higher than that of the newly read ch operator
    CrtSubtree( t, c);
//Construct a subtree based on the stacks top element
    Pop(S, c);
//Pop the top element from the stack
  }
if ( ch!= '#' ) Push( S, ch);
//Push the ch operator onto the stack
break;
```

(3) Construct a binary tree from its pre-order and in-order sequences

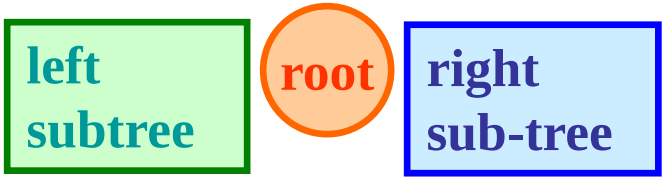
Given the preorder sequence abcdefg of a binary tree, can we uniquely determine the tree structure?

cannot

What if the inorder sequence of a binary tree is known as "cbdaegf"?

Preorder sequence of a binary tree

Binary tree in-order sequence

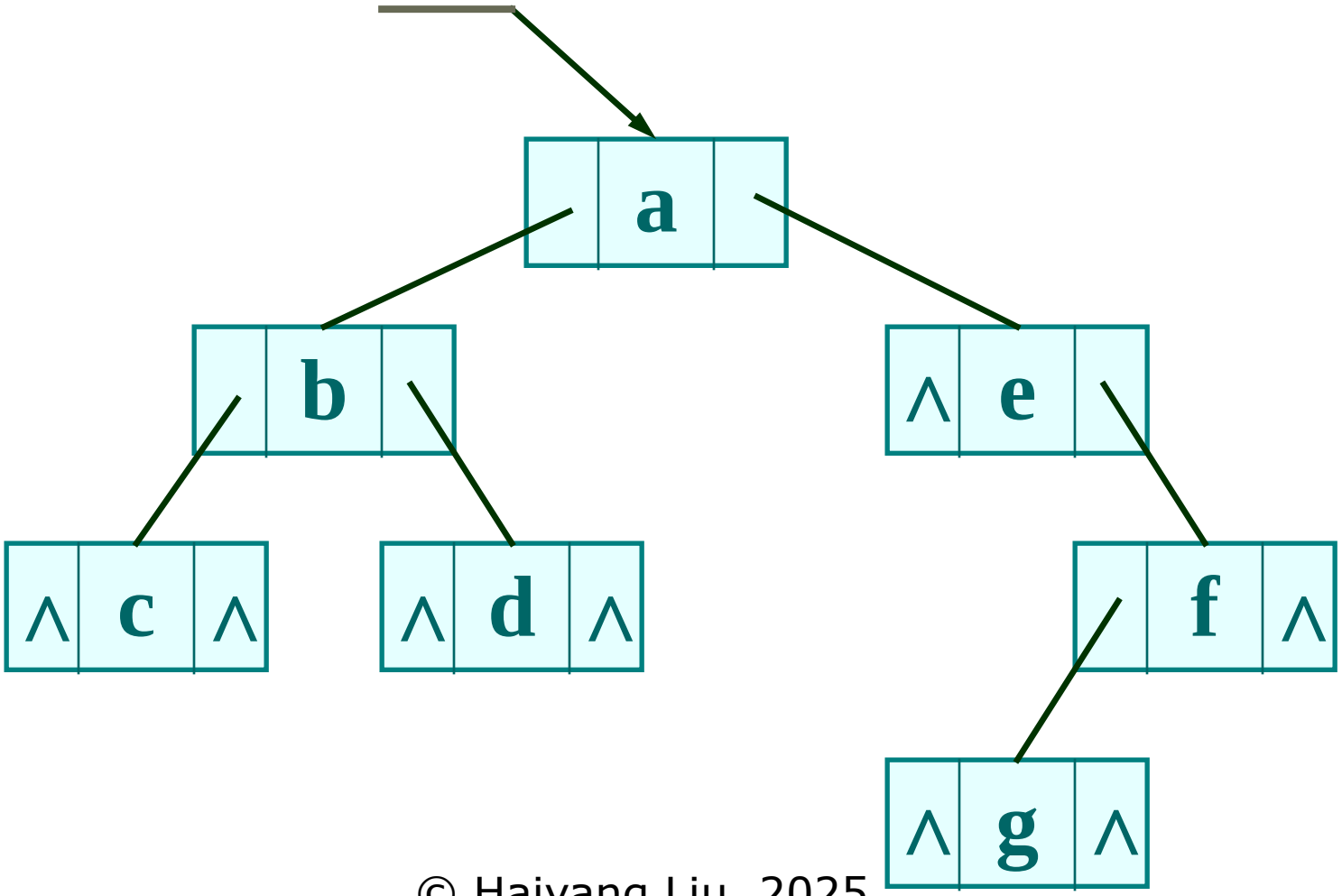


© Haiyang Liu, 2025

Example:

a b c d e f g — Preorder sequence

c b d a e g f — Inorder sequence



```
void CrtBT(BiTree& T, char pre[], char ino[],
                                  int ps, int is, int n ) {
//Given pre[ps..ps+n-1] as the preorder sequence of a binary tree,
// ino[is..is+n-1] represents the inorder sequence of a binary tree. This algorithm constructs a binary linked list using these two sequences, where ps denotes the starting position of the preorder sequence, "is" is the starting position of the inorder sequence, and n is the sequence length.
  if (n==0) T=NULL;
  else {
     k=Search(ino, pre[ps]); //Query the predecessor sequence
//The position of the first character in the middle sequence
     if (k== -1)  T=NULL;
     else {   … …      }
  }
} // CrtBT
```

```
T=(BiTNode*)malloc(sizeof(BiTNode));
T->data = pre[ps];        //Establish the tree root
if (k==is)  T->Lchild = NULL; //If the first character in the preorder sequence is also the first character in the inorder sequence, it indicates that there is no left subtree.
else  CrtBT(T->Lchild, pre[], ino[],
                                ps+1, is, k-is );
if (k==is+n-1) T->Rchild = NULL; //If the first character in the preorder sequence is the last character in the inorder sequence, it indicates that there is no right subtree.
else  CrtBT(T->Rchild, pre[], ino[],
                ps+1+(k-is), k+1, n-(k-is)-1 );
```

ps↓

Preorder: a<u>bcd</u> <u>efg</u> (ps..ps+n-1)

is↓ ↓k

Inorder: <u>cbd</u>a <u>egf</u> (is..is+n-1)

6. Construct a binary tree from the traversal sequence

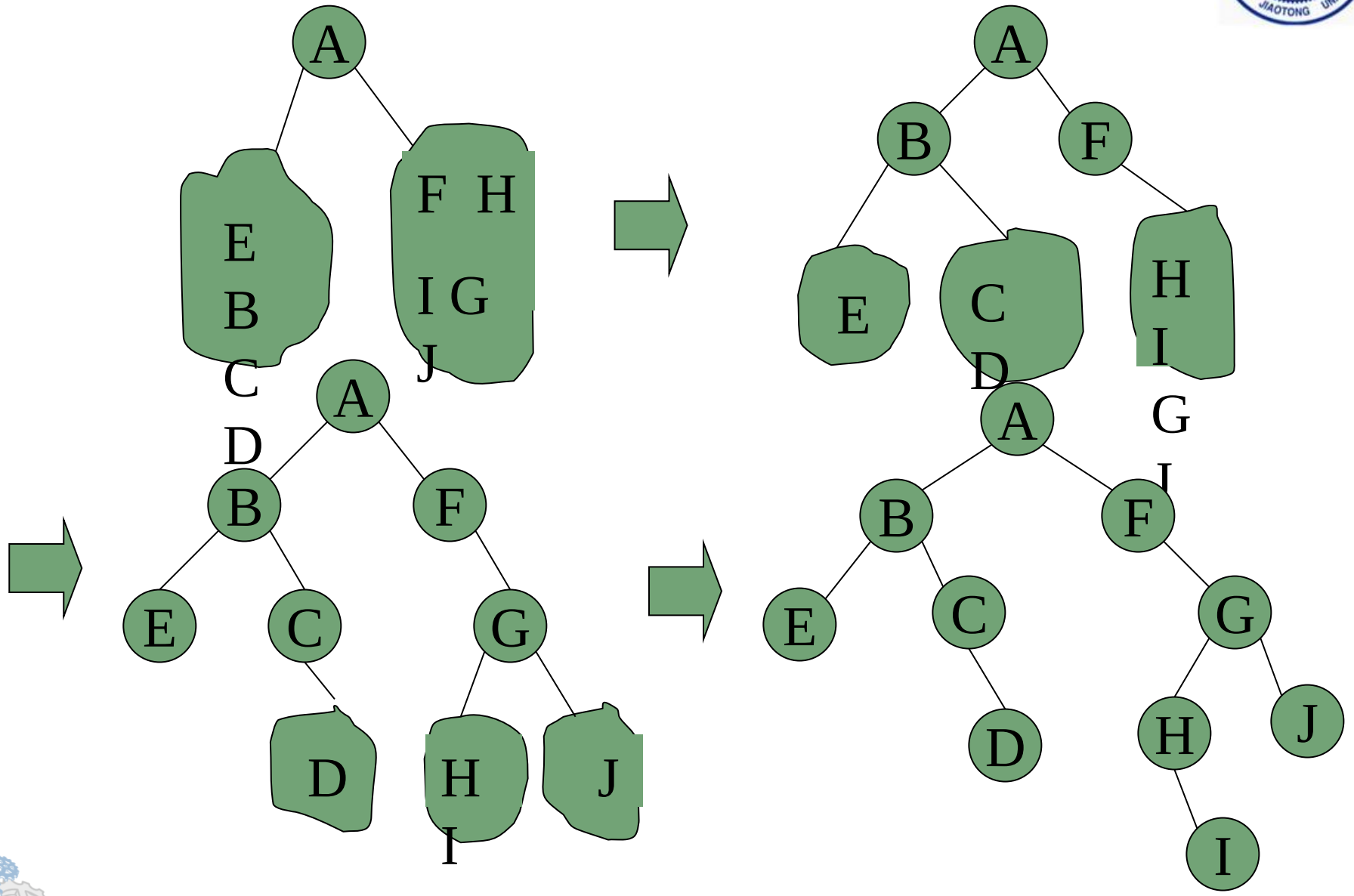
Given the preorder and inorder sequences of a binary tree, the unique binary tree can be constructed.

Example: Given a binary tree with
the pre-order sequence ABECDFGHIJ,
the in-order sequence EBCDAFHIGJ,
construct the tree.

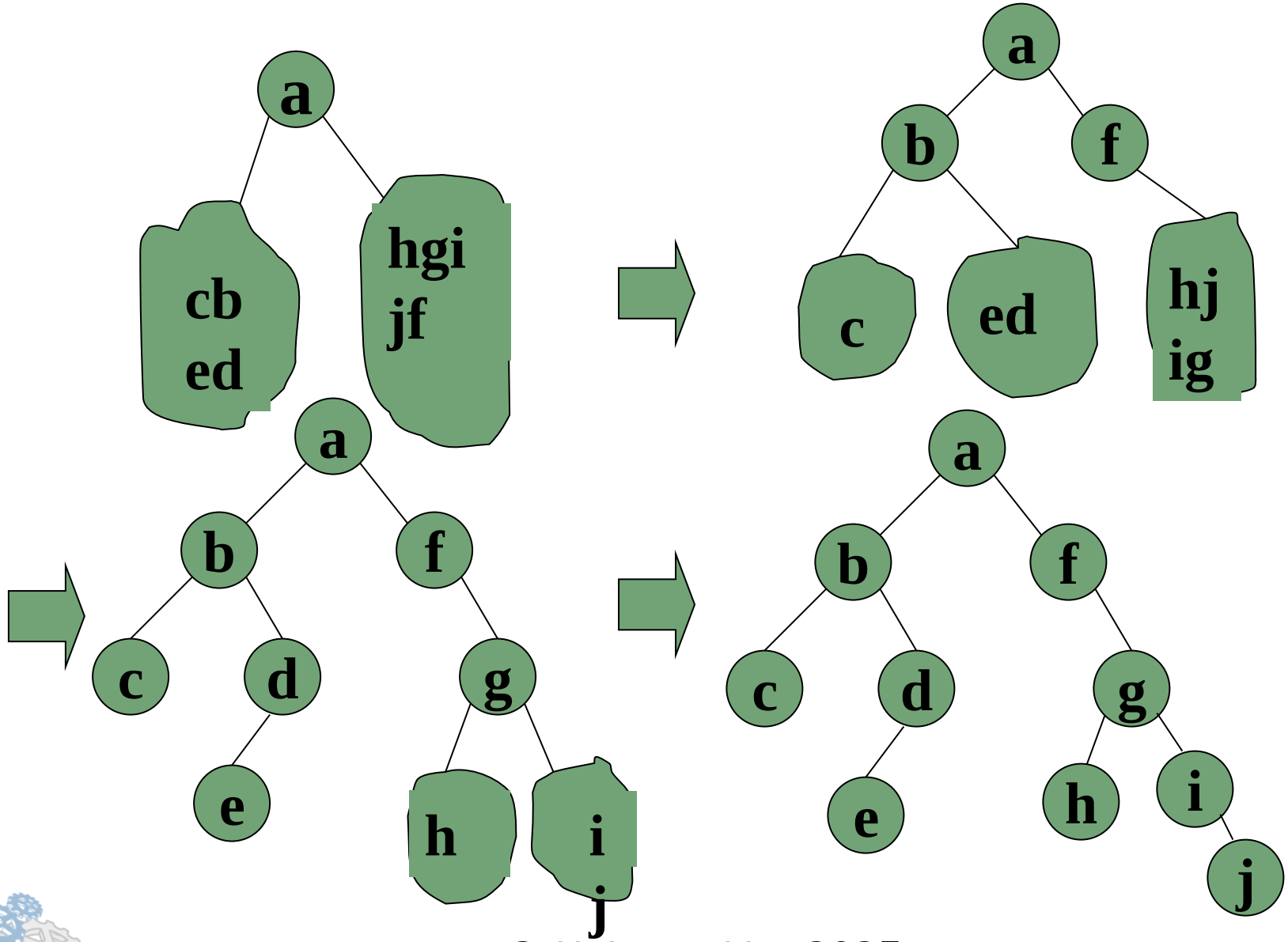
© Haiyang Liu, 2025

Given: The pre-order traversal sequence is ABECDFGHIJ

The in-order traversal sequence is EBCDAFHIGJ



Given: the inorder traversal sequence is cbedafhgij
The post-order traversal sequence iscedbhjigfa



© Haiyang Liu, 2025

Main content

- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- **Threaded binary tree**
- Trees and forests
- Tree and forest traversal
- Huffman tree and Huffman coding

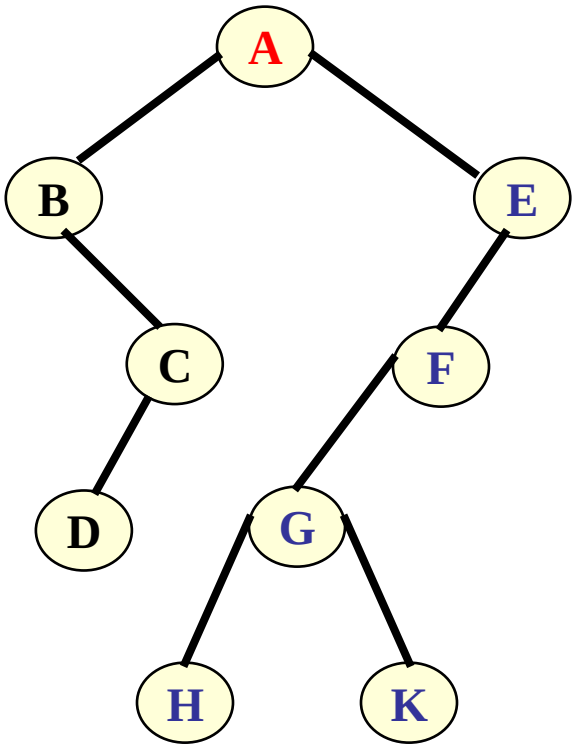
Threaded binary tree

- What is a clue binary tree?
- Traversing a linked list
- How to create a lead list?

© Haiyang Liu, 2025

What is a threaded binary tree?

- When the binary linked list is used as the storage structure, only the left and right children of the node can be found, but the predecessor (precedent) and successor of the node in any sequence can not be obtained directly. This information can only be obtained in the dynamic process of traversal.



Can we save the information of the predecessor and successor of the node obtained during the traversal?

© Haiyang Liu, 2025

Method 1:

Two pointer fields, fwd and bkwd, are added to each node to indicate the predecessor and successor information obtained during sequential traversal. This significantly reduces the storage density of the structure.

Method 2:

In a binary linked list with n nodes, there must be n+1 null links. These null links are used to store the predecessor and successor information of the nodes.

Add two flag fields, LTag and RTag, to the nodes of the binary linked list.

lchild	ltag	data	rtag	rchild

Node Structure

Among them:

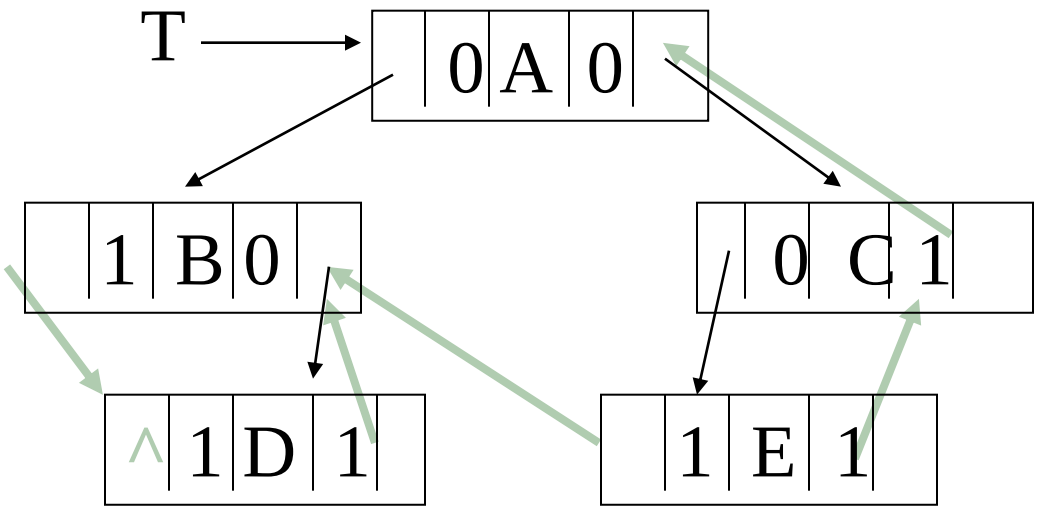
ltag=
- **The 0 child field indicates the left child of the node**
- **The 1 child field indicates the predecessor of the node**

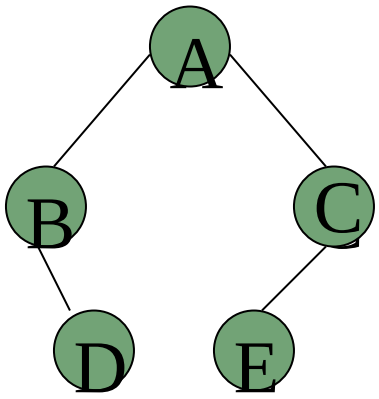
rtag=
- **The 0 child field indicates the right child of the node**
- **The 1 child field indicates the successor node**

A pointer to the "predecessor" and "successor" in a linear sequence is called a "thread".

The storage structure of a binary tree defined in this way is called a "threaded list".

Example:Postorder threaded binary tree DBECA





This type of binary tree with thread pointers is called a threaded binary tree.

The process of traversing a binary tree in a certain order to make it a threaded binary tree is called threading.

Description of the trace list type:

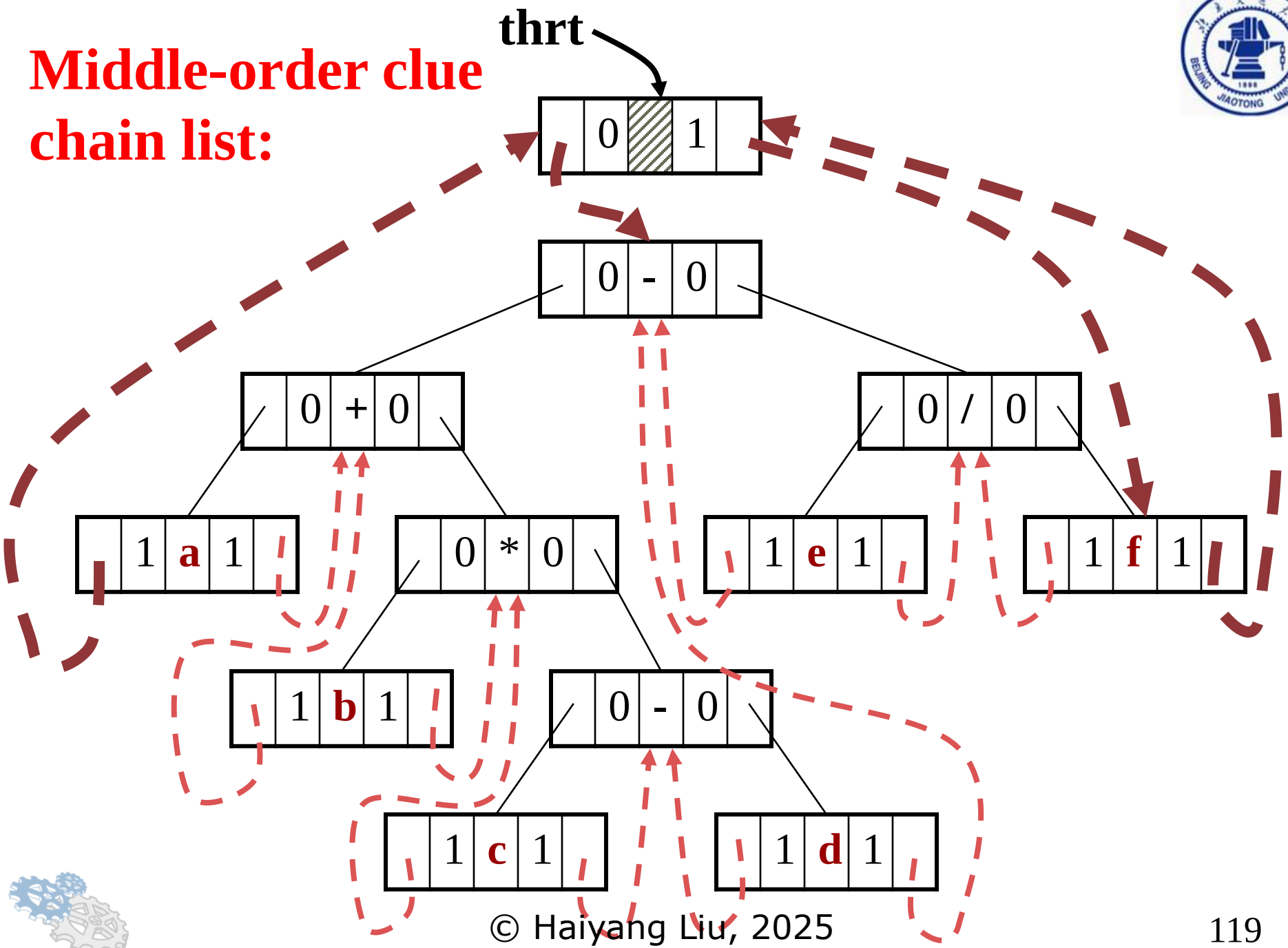
```
typedef enum { Link, Thread } PointerThr;
                    // Link==0: pointer, Thread==1: thread


typedef struct BiThrNod {
    TElemType          data;
    struct BiThrNode  *lchild, *rchild;  // left and right pointers
    PointerThr         LTag, RTag;       //  Left and right flags
} BiThrNode, *BiThrTree;
```

© Haiyang Liu, 2025

The head node of the threaded linked list of a binary tree

For convenience, following the storage structure of linear lists, a head node is added to the linked list of a binary tree. The lchild field of the head node points to the root node of the binary tree (note that a binary tree represented by a linked list is given its head pointer, which refers to the head node). The rchild field of the head node points to the last node visited during in-order traversal. In the in-order sequence of the binary tree, both the lchild field of the first node and the rchild field of the last node point to the head node.

Middle-order clue chain list:



© Haiyang Liu, 2025

2. Traverse the threaded list

Traverse a linked list with a middle-order clue

※ **The first node in the middle sequence?**

The node at the **bottom-left** of the left subtree (if the left subtree does not exist).

※ **What is the successor of a node in a singly linked list?**

If there is no right subtree, it is the node pointed by the successor thread;

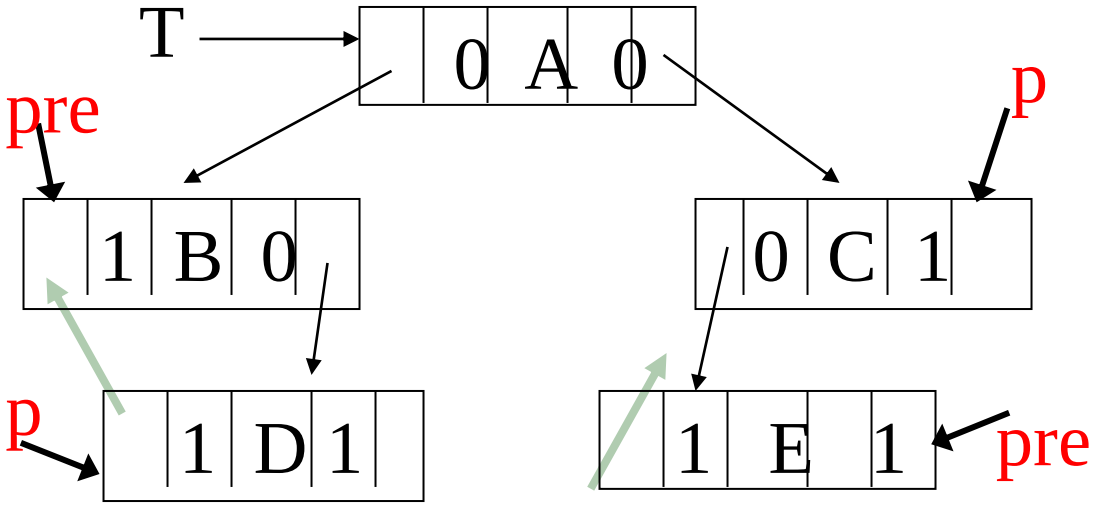
Otherwise, it is the first node visited during a mid-order traversal of its right subtree.

© Haiyang Liu, 2025

```
void InOrderTraverse_Thr(BiThrTree T,
                         void (*Visit)(TElemType e)) {
    // T points to the head node, and the left link of the head node, lchild,
points to the root node
        // Non-recursive algorithm for mid-traversal of binary clue tree T,
p = T->lchild;      //  p points to the root node
while (p != T)      // When the empty tree or traversal ends, p==T
 { while (p->LTag==Link)     p=p->lchild; // The first node
   if(!visit(p->data)) return error;  //Accessing nodes with empty left
subtrees
   while (p->RTag==Thread && p->rchild!=T)
      { p = p->rchild;  Visit(p->data); } // Visit the successor node
   p = p->rchild;                          // p advances to its right child root
  }
} // InOrderTraverse_Thr
```

How to build a threaded list of clues?

- The process of threading is the process of modifying the null pointer during traversal.
- To record the order of nodes visited during traversal, a pointer pre is attached to the predecessor of the node currently being visited, which is pointed to by pointer p.



Inorder： BDAEC

```
void  InThreading ( BiThrTree p)
{if (p) // Tracing the middle order for a non-empty binary tree with p as its root
{InThreading (p-> lchild); // Left subtree cycled
  if ( !p-> lchild)// Left child is empty, build the predecessor clue
     { p->LTag = Thread;    p->lchild = pre; }
  if  ( !pre-> rchild) // The left child preceding it is empty, establishing the successor link
     {  pre->RTag = Thread;   pre->rchild = p;  }
    pre = p; // keeps the predecessor of the pre pointing to p
    InThreading (p-> rchild); // Right subtree lenticulation
} //  if
} //  InThreading
```

```
Status InOrderThreading (BiThrTree &Thrt, BiThrTree T)
  // // Build a middle-order linked list Thrt points to the
header node, and T is the root of the binary tree
 { if (!(Thrt = (BiThrTree)malloc(
                                  sizeof( BiThrNode))))
     exit (OVERFLOW);          // Allocate the head node
  Thrt->LTag = Link;  Thrt->RTag =Thread;

          …    …
  Thrt->rchild = Thrt;
// The right pointer of the head node is set to the child
  return OK;
} // InOrderThreading
```

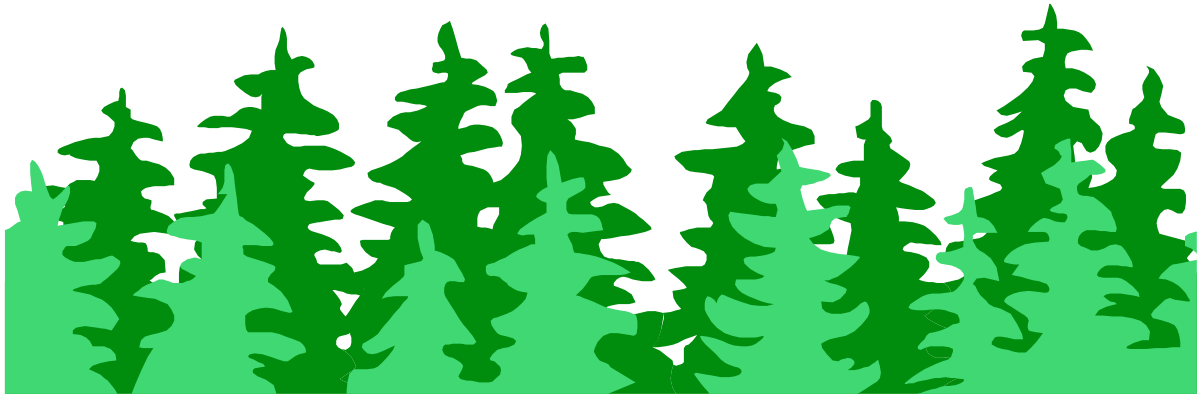
```
if ( !T)   Thrt-> lchild =  Thrt;
//If the binary tree is empty, the left pointer returns
else {
        Thrt-> lchild = T;   pre =  Thrt;
        InThreading (T);
        // In-order traversal with in-order threading
        prev-> right child = Thrt; // process the last node
        pre-> RTag = Thread;
        Thrt-> rchild = pre;
}
```

Main content

- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- Threaded binary tree
- **Trees and forests**
- Tree and forest traversal
- Huffman tree and Huffman coding

Trees and Forests

- Tree storage structure
- Tree, forest, and binary tree conversion



1. Tree storage structure

1. Parental notation (sequential or chain)

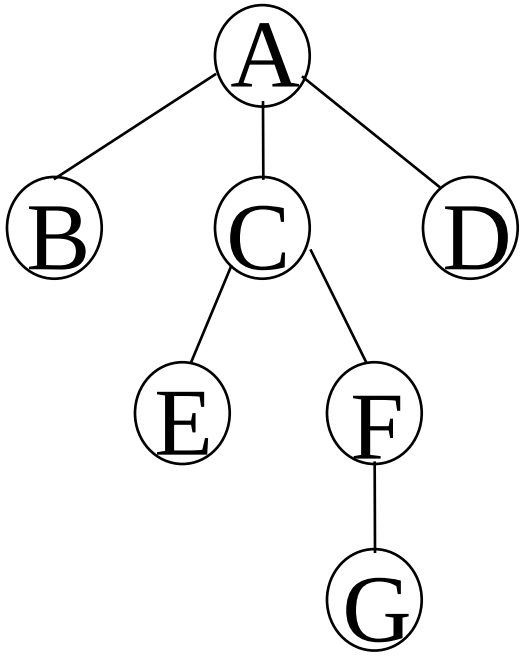
2. Child Expression

3. Child linked list representation (adjacent list)

4. Tree binary linked list (children-branches) notation

1. Parental expression:

A set of nodes in a continuous space storage tree, with an indicator attached to each node indicating the location of its parent node.



Array index data parent

0	A	-1
1	B	0
2	C	0
3	D	0
4	E	2
5	F	2
6	G	5

© Haiyang Liu, 2025

C language type description:

data	parent

```
#define MAX_TREE_SIZE  100

typedef struct PTNode {  //Node structure
   Elem  data;
   int    parent;            // Parent position field
} PTNode;

typedef struct {              //Tree
     PTNode  nodes[MAX_TREE_SIZE];
     int    r, n;          // root node position and node count
   } PTree;
```

© Haiyang Liu, 2025

This storage structure leverages the property that each node (except the root) has a unique parent. The **PARENT(T, x)** operation can be executed in constant time. By repeatedly calling PARENT operations until encountering a node with no parent (i.e., a parent value of-1), the root of the tree is identified. This constitutes the execution process of the **ROOT(x)** operation.

However, in this representation, you need to traverse the entire structure to find the children of a node.

2. Childrens expressions:

Since a tree may contain multiple subtrees at each node, a linked storage structure can utilize a doubly linked list. Each node contains multiple pointer fields, with each pointer pointing to the root of a subtree. In this configuration, the nodes in the linked list can adopt two distinct formats:

data	child1	child2	…	childd

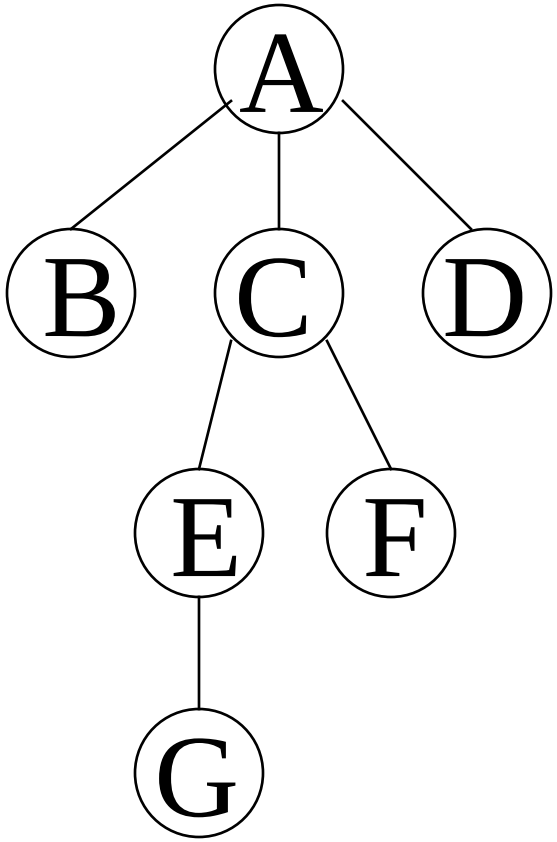
data	degree	child1	child2	…	Childd

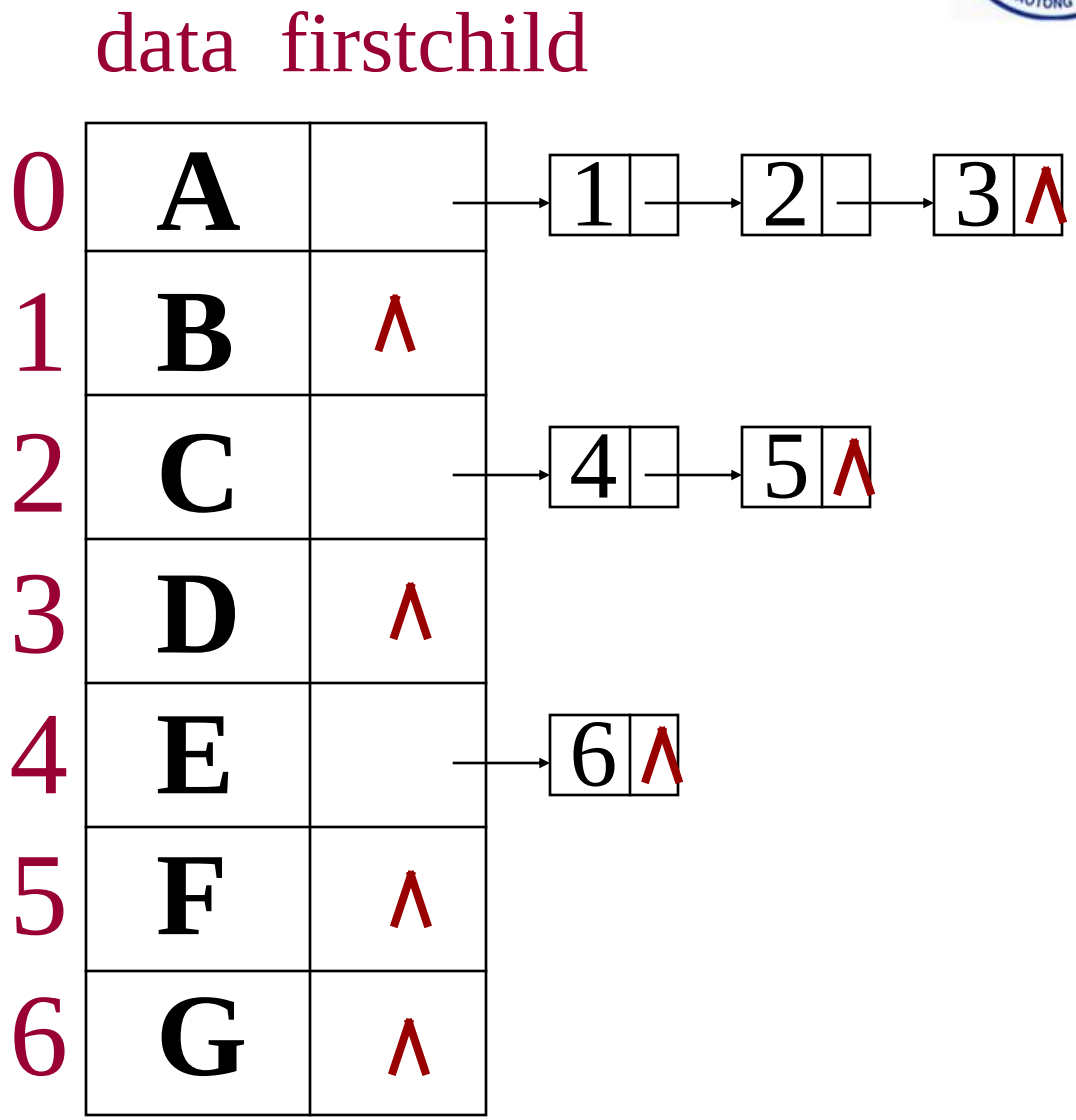
When using the first node format, the nodes in a doubly linked list are isomorphic, where d represents the degree of the tree. Since many nodes in the tree have degrees less than d, the linked list contains numerous empty fields, resulting in space wastage. It can be easily deduced that in a tree with n nodes of degree k, there must be n(k-1)+1 empty fields.

The second node format creates distinct nodes in a doubly linked list, where d represents the degree of the node and the degree field matches d. While this saves storage space, it makes operations less convenient.

3. Child list representation:

Alternatively, we can arrange the child nodes of each parent node into a linear list, using a singly linked list as the storage structure. This results in n child lists for each parent node (with empty child lists for leaf nodes). The n head pointers form another linear list, which can be stored sequentially for efficient searching.





© Haiyang Liu, 2025

C Language Type Description (Three Parts):

Child node structure:

child	next

```
typedef  struct  CTNode {
int          child;
struct  CTNode *next;
} * ChildPtr;
```

Parent-child node structure:

data	firstchild

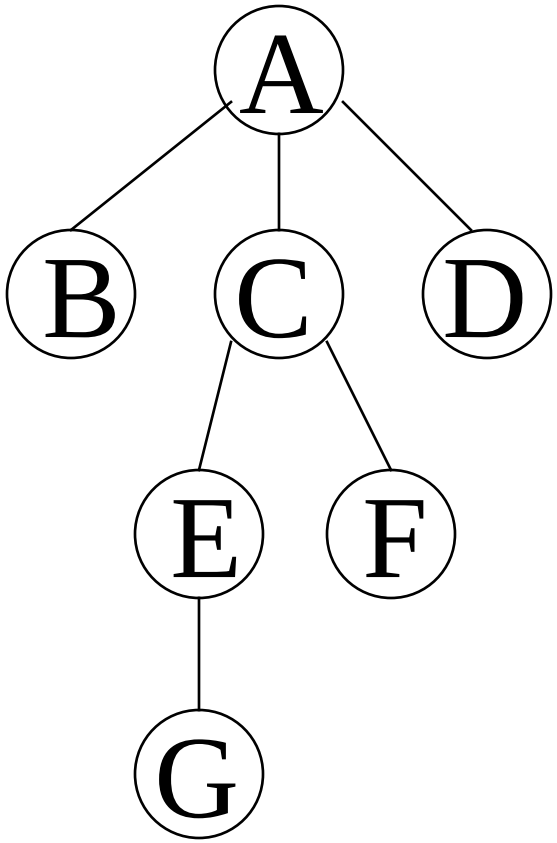
```
typedef  struct {
Elem    data;
ChildPtr firstchild; //Head pointer of the child chain
}  CTBox;
```

tree structure：

```
typedef  struct {
CTBox  nodes[MAX_TREE_SIZE];
int n, r; // Number of nodes and root node position
}  CTree;
```

Unlike the parent-child representation, the child representation facilitates operations involving children but is not suitable for PARTEN(x) operations. A parent-child child list can be created by combining the parent-child representation with the child list.

List of children with both parents



	data	parent	firstchild
0	A	-1	→ 1 → 2 → 3 ∧
1	B	0	∧
2	C	0	→ 4 → 5 ∧
3	D	0	∧
4	E	2	→ 6 ∧
5	F	2	∧
6	G	4	∧

© Haiyang Liu, 2025

4. Binary tree linked list storage representation

- Binary tree linked list storage representation: Child-Parent representation
- Also known as binary tree representation or binary linked list representation, this method uses a binary linked list as the storage structure for the tree. Each node in the linked list contains two fields: one pointing to its first child node and the other to its next sibling node.

C language type description:

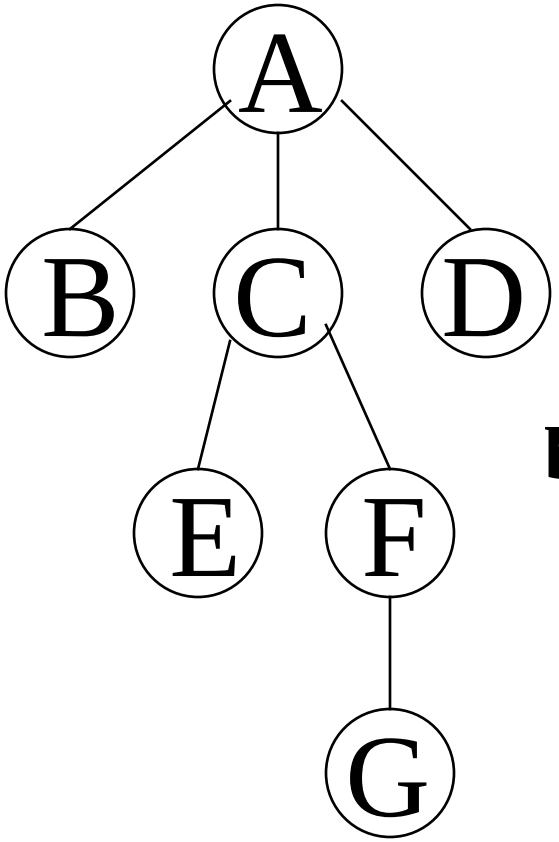
Node structure:

firstchild	**data**	**nextsibling**

```
typedef struct CSNode{
    Elem        data;
    struct CSNode
        *firstchild, *nextsibling;
} CSNode, *CSTree;
```

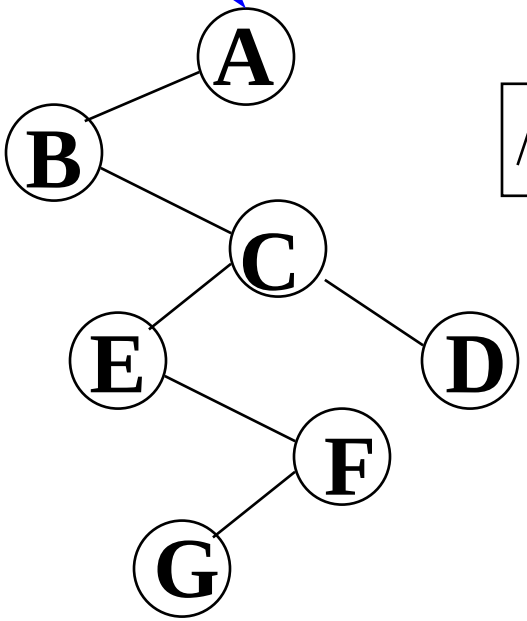
© Haiyang Liu, 2025

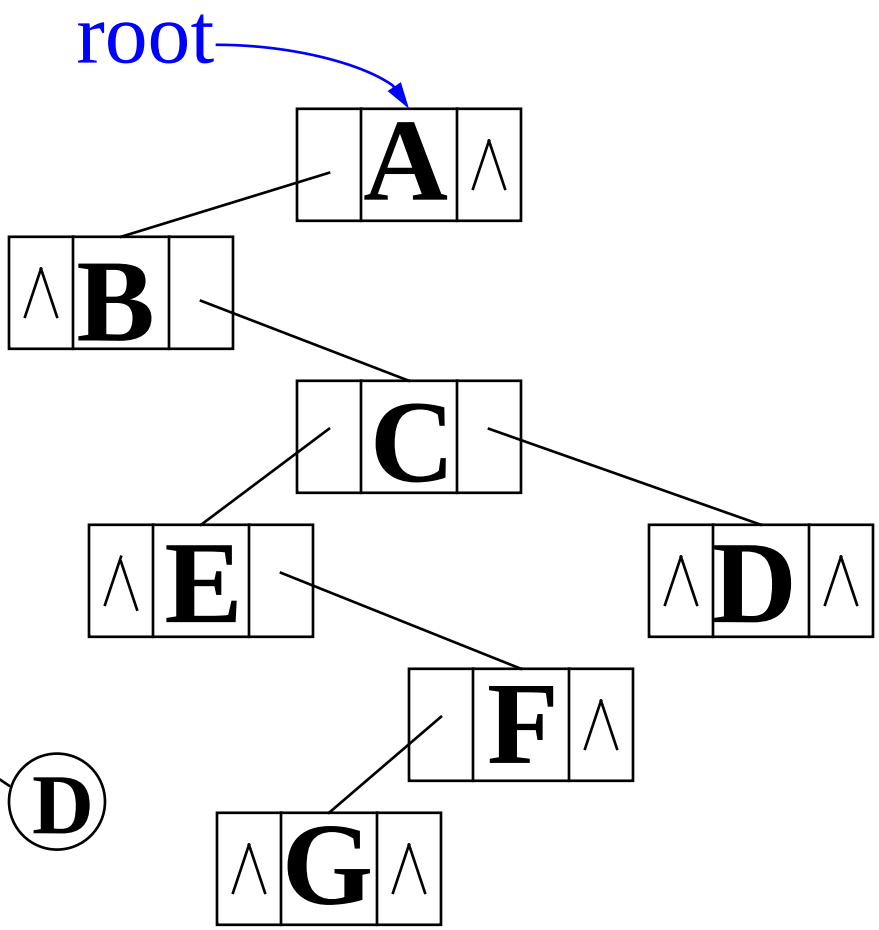
Binary linked list:



tree

binary tree





© Haiyang Liu, 2025

The following conclusions can be drawn from the above figure:

Given a tree, there is only one binary tree that corresponds to it. In terms of storage structure, the binary linked list representation of a tree is identical to that of its corresponding binary tree. The only difference lies in the interpretation: the left and right subtrees are redefined as children and siblings respectively.

In a binary tree corresponding to any tree, the right subtree must be empty.

This storage structure facilitates various tree operations (implemented through binary tree operations). It makes it easy to perform operations such as locating a nodes children.

For example, to access the i-th child of node x, simply locate the firstchild in the firstchild field, then traverse i-1 steps along the nextsibling field of the child nodes to reach the i-th child of x.

II. Tree, forest, and binary tree conversion

The correspondence between a forest and a binary tree

Set the forest

F = (T 1 , T 2 , …, T n);

T 1 = (root, t 11 , t 12 , …, t 1m);

binary tree

B =(LBT, Node(root), RBT);

The conversion rule from forest F (T1, T2,..., Tn) to binary tree B is:

If F = Φ, then B = Φ;

otherwise ,

B (root) is obtained by ROOT (T1);

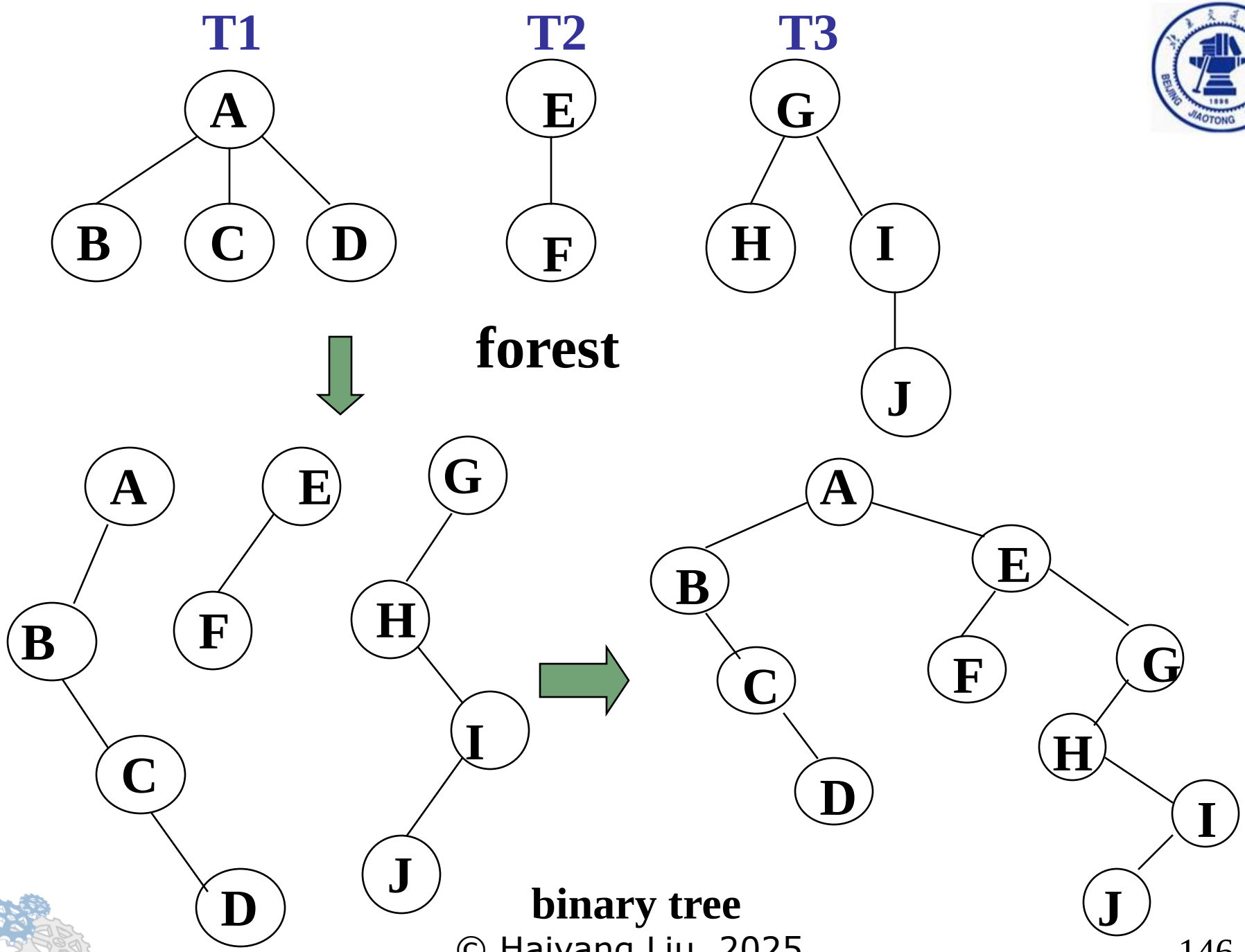
The binary tree corresponding to (t 11, t 12,..., t 1m)

Get the LBT of the entire binary tree;

A binary tree corresponding to the transformation of (T2, T3,..., Tn)

Get the RBT of the entire binary tree.

© Haiyang Liu, 2025



© Haiyang Liu, 2025

The conversion rules from a binary tree to a forest are:

If B = Φ, then F = Φ;

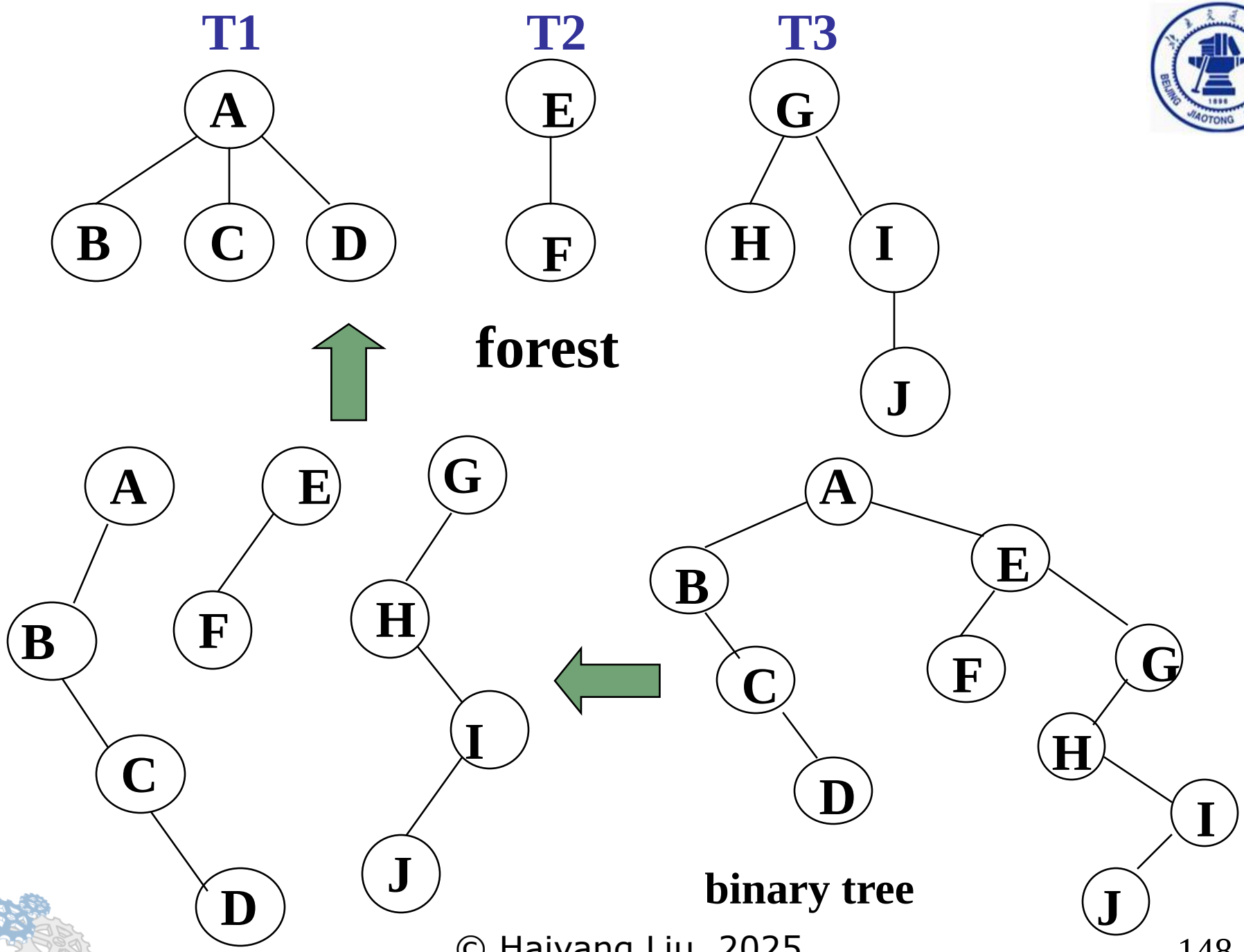
otherwise ,

ROOT (T_1) is obtained by B (root);

Corresponding to the conversion by LBT, (t_{11}, t_{12},..., t_{1m}) is obtained;

Corresponding to the RBT transformation, (T_2, T_3,..., T_n) is obtained.

© Haiyang Liu, 2025



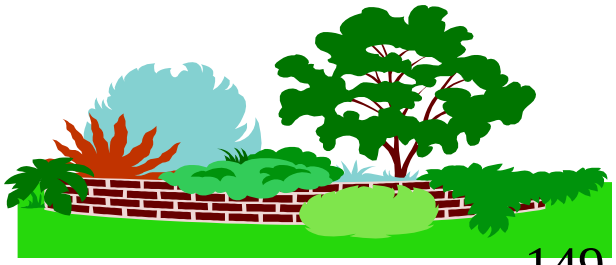
© Haiyang Liu, 2025

6.7 Trees and forest traversals

I. Tree traversal

II. FOREST TRAVEL

III. Applications of tree traversal



Main contents

- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- Threaded binary tree
- Trees and forests
- **Tree and forest traversal**
- Huffman tree and Huffman coding

© Haiyang Liu, 2025

I. Tree traversal There are three search paths:

First root (sequence) traversal:

If the tree is not empty, then visit the root node first, and then visit each subtree in turn. (This is equivalent to the pre-order traversal of a corresponding binary tree.)

Back root (in order) traversal:

If the tree is not empty, then traverse each subtree in sequence from the root, and then visit the root node. (This is equivalent to the middle order traversal of a corresponding binary tree.)

Browse by hierarchy:

If the tree is not empty, visit each node in the tree from top to bottom and left to right.

The order of visiting the vertices from the root to the top is:

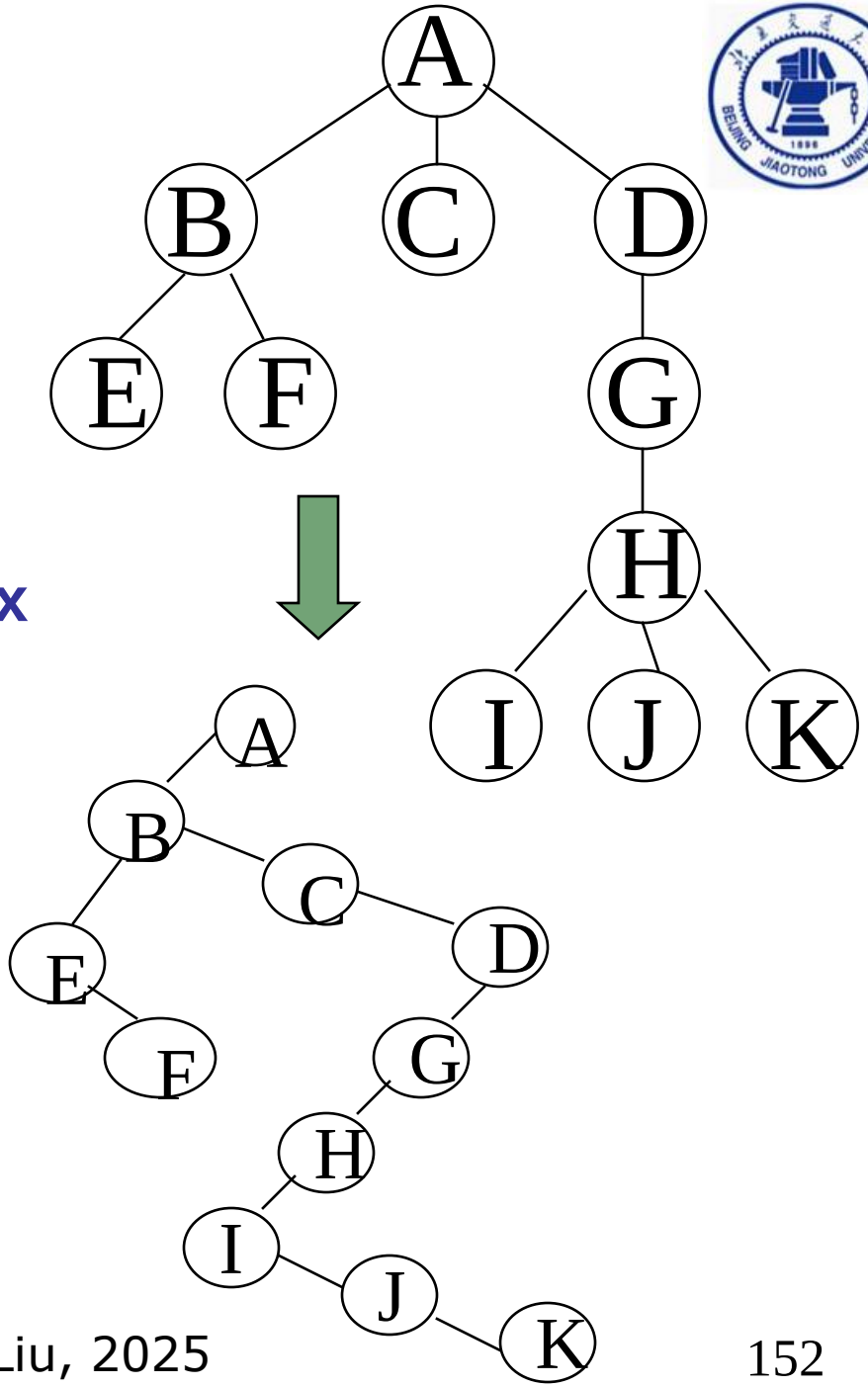
A B E F C D G H I J K

The order of visiting the vertex at the end of the root is:

E F B C I J K H G D A

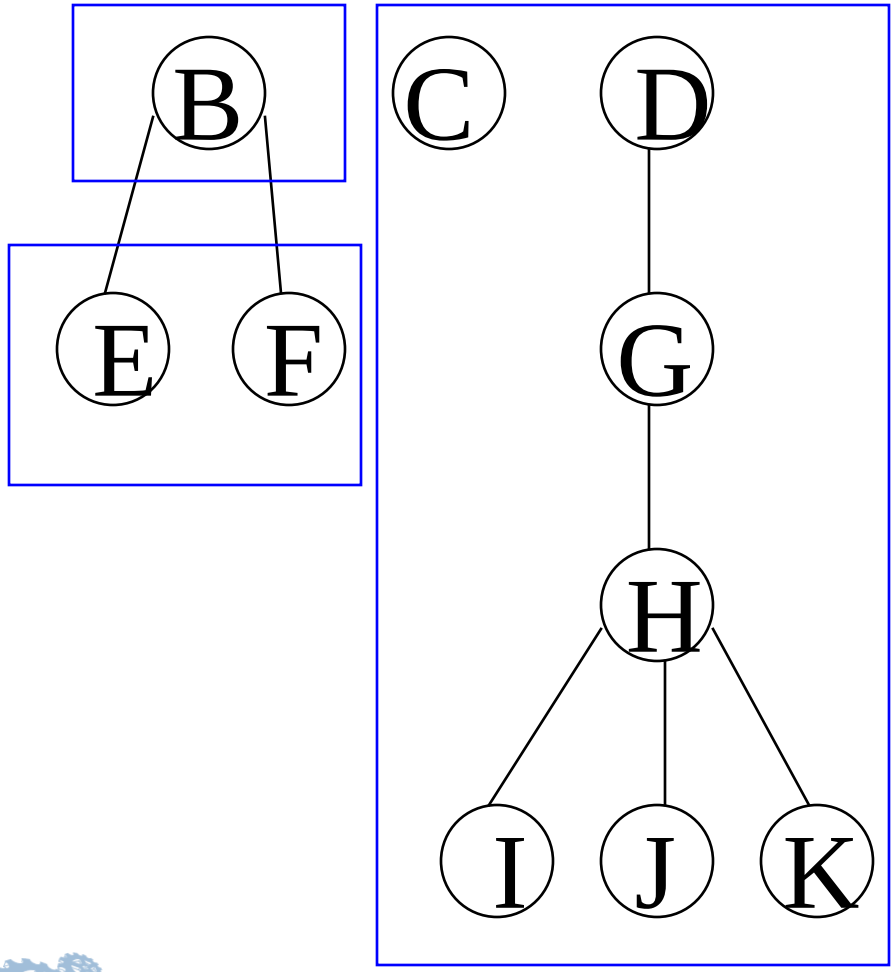
The order in which vertices are visited during a hierarchical traversal:

A B C D E F G H I J K



© Haiyang Liu, 2025

II. Forest Walk



The forest consists of three parts:

1. The root node of the first tree in the forest;

2. Forest of the first trees subtrees in the forest;

3. A forest composed of other trees.

© Haiyang Liu, 2025

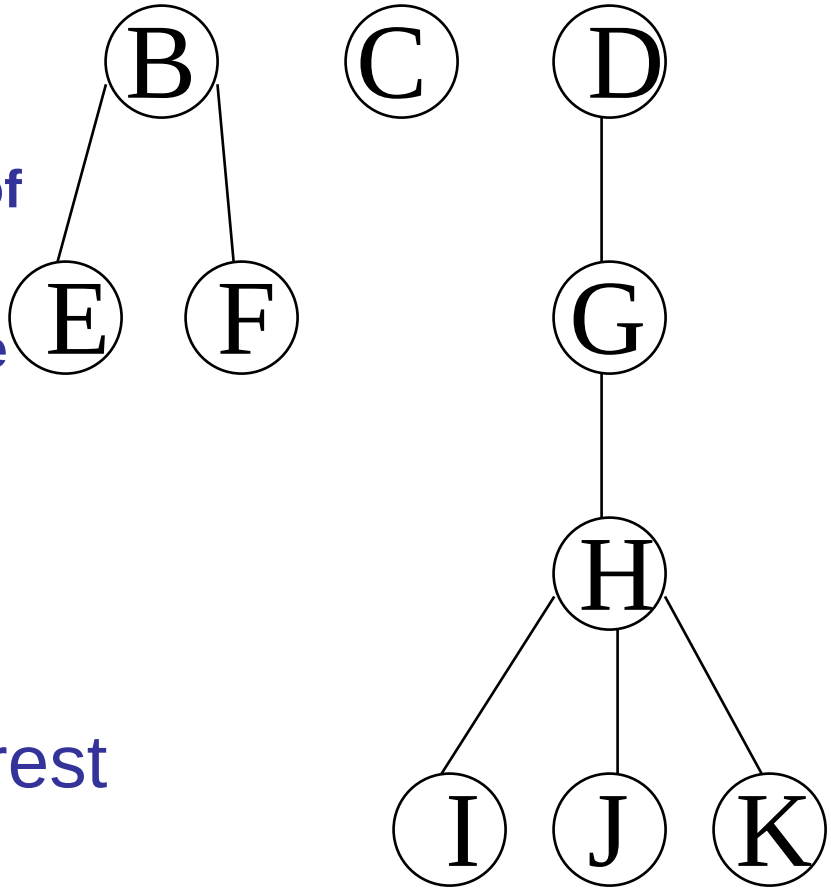
1.Preorder traversal

If the forest is not empty:

Visit the root node of the first tree in the forest;

Preorder traverse the forest of subtrees of the first tree in the forest;

Preorder traverse the forest formed by the remaining trees (except the first tree).



That is, perform **a pre-root traversal** of each tree in the forest **from left to right.**

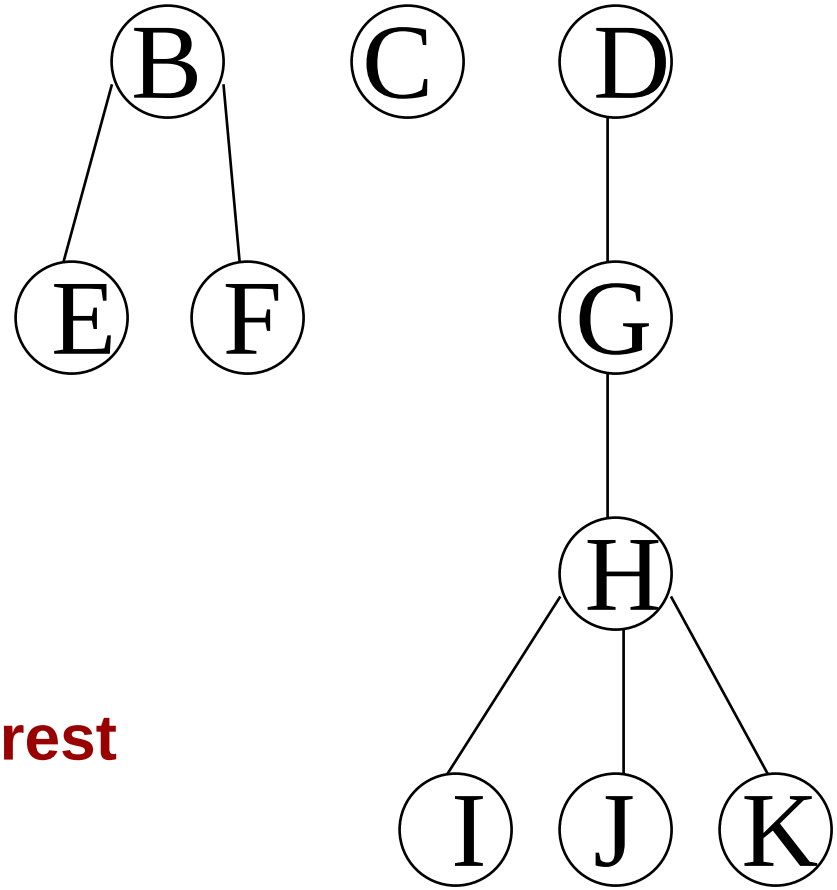
2. In-order traversal

If the forest is not empty, then:

Perform an in-order traversal of the subtree forest of the first tree in the forest;

Visit the root node of the first tree in the forest;

Perform an in-order traversal of the forest composed of the remaining trees in the forest.



That is, traverse each tree **in the forest from left to right in post-order.**

What is the correspondence between tree traversal and forest traversal and binary tree traversal?

tree	forest	binary tree
Traverse all roots	Preorder traversal	Preorder traversal
Post-order traversal	inorder traversal	inorder traversal

© Haiyang Liu, 2025

3. Application of tree traversal

The tree is stored using a child-branch linked list (binary linked list).

```
typedef struct CSNode{
Elem        data;
struct CSNode * firstchild, *nextsibling;
} CSNode, *CSTree;
```

1. Query tree depth

2. Output all paths from the root to the leaves in the tree

self-study

3. Storage structure of the built-in

1. Algorithm for finding the depth of a tree:

```
int TreeDepth(CSTree T) {
  if(!T)  return 0;
  else {
    h1 = TreeDepth( T->firstchild );
    h2 = TreeDepth( T->nextsibling);
    return(max(h1+1, h2));
  }
} // TreeDepth
```

© Haiyang Liu, 2025

Main content

- Tree type definitions
- Type definition of binary tree
- Properties of a binary tree
- Storage structure of binary tree
- binary tree traversal traversal of binary tree
- Threaded binary tree
- Trees and forests
- Tree and forest traversal
- **Huffman tree and Huffman coding**

Huffman tree and Huffman coding

- Definition of optimal tree
- How to construct an optimal tree
- Prefix coding

I. Definition of optimal tree

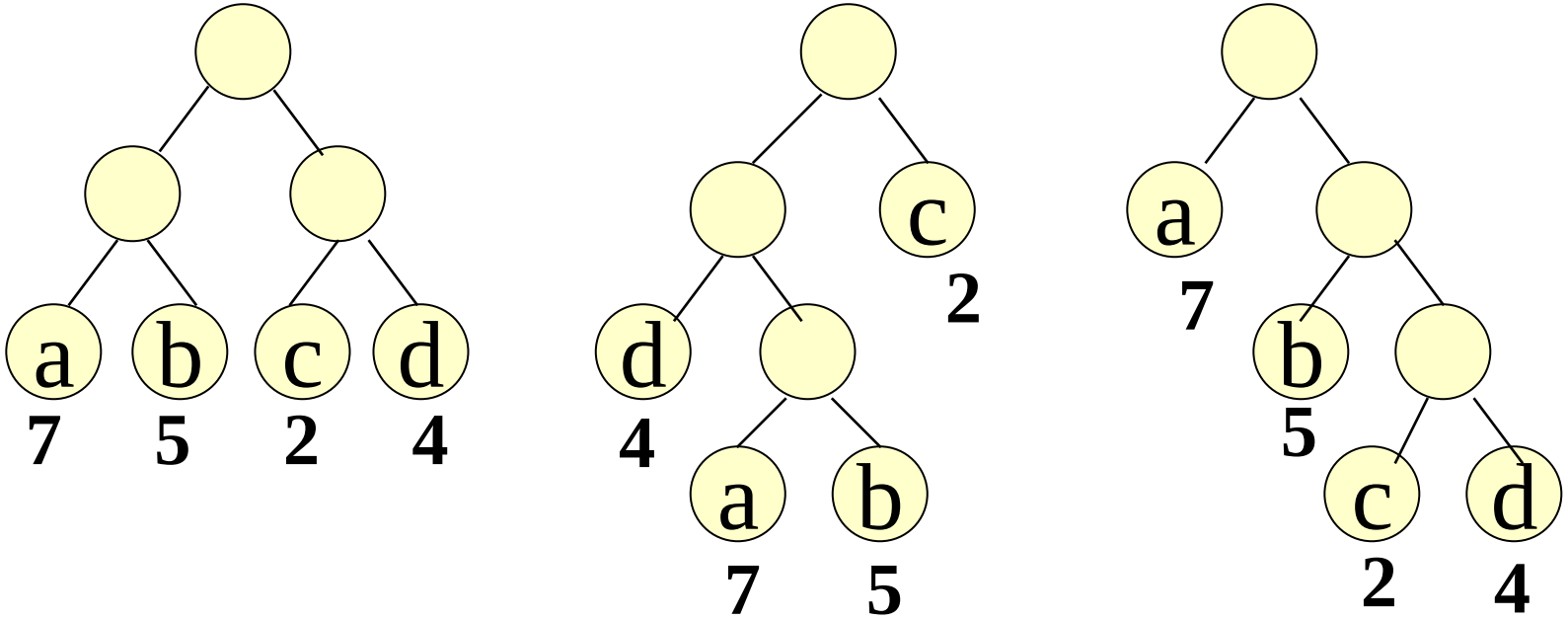
- **way：**
  The branch between one node and another in a tree forms the path between those two nodes.

- **Path length of nodes:**
  The number of branches on the path from the root node to that node.

- **The path length of the tree:**
  The sum of the path lengths of each node in the tree.

Trees weighted path length: $WPL(T) = \Sigma w_k l_k$

The sum of the weighted path lengths of all leaf nodes in the tree.



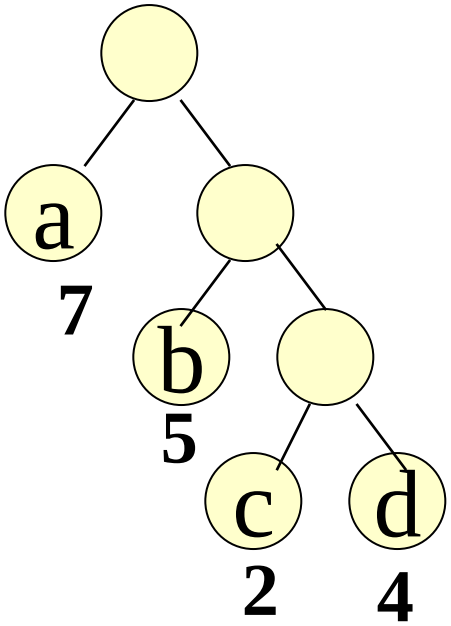
(1) **WPL=7*2+5*2+2*2+4*2=36**

(2) **WPL= 7*3+5*3+2*1+4*2=46**

(3) **WPL= 7*1+5*2+2*3+4*3=35**

- **Optimal tree:**

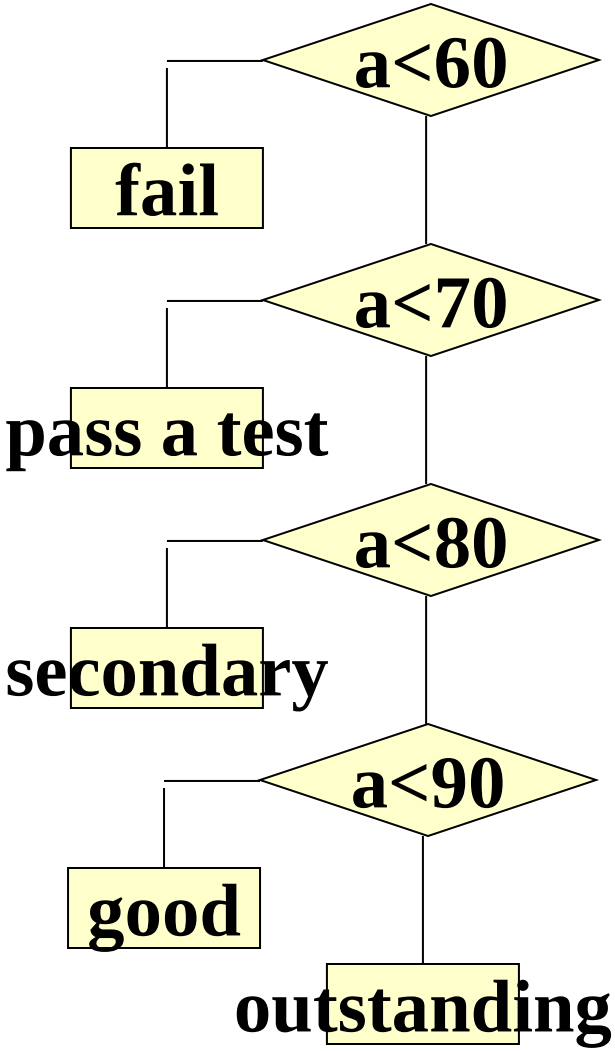
  Among all m-ary trees with n identical leaf nodes and weights, there exists one with **the minimum weighted path length,** known as the **optimal tree or Huffman tree.**



WPL= 7*1+5*2+2*3+4*3=35

When solving a problem, Huffman tree can be used to obtain the optimal decision algorithm.

Example: Convert a percentage to a five-point scale. The decision tree used in the judgment process:

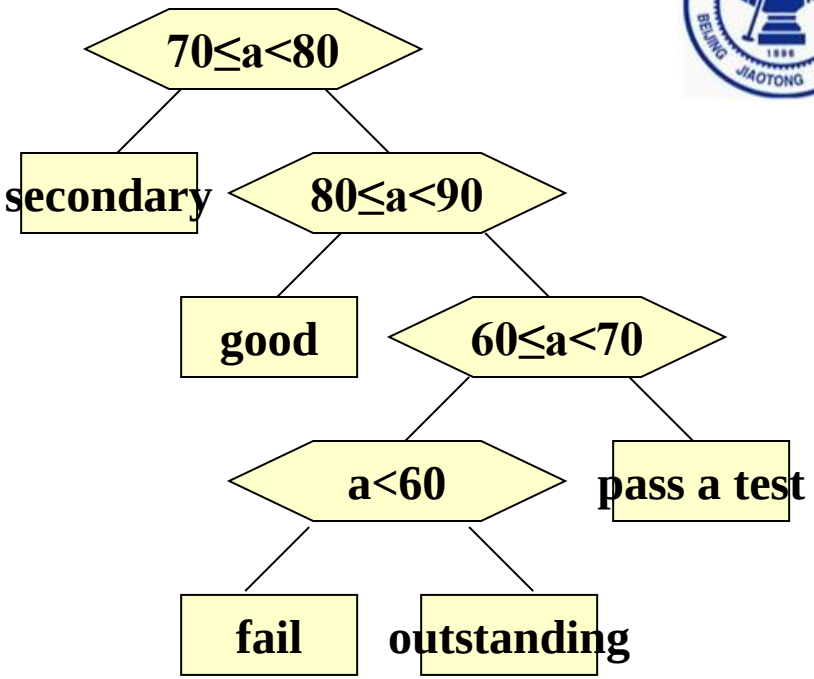


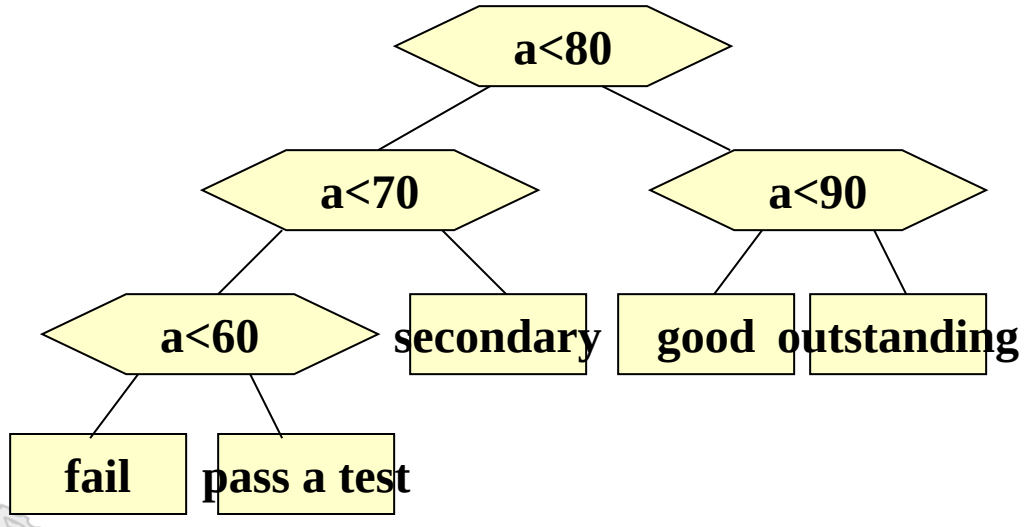
When implementing the aforementioned algorithm repeatedly with substantial input data, its computational efficiency becomes critical. In practical applications, student performance scores are unevenly distributed across five tiers. As shown in the table below, over 80% of data points require three or more comparisons to yield definitive results.

fraction	0~59	60~69	70~79	80~89	90~100
scale	0.05	0.15	0.40	0.30	0.10

© Haiyang Liu, 2025

A Huffman tree with five leaf nodes is constructed based on the probability of students scores being distributed in five grades, and the diagrammatic judgment process is obtained, which enables most data to get the result with fewer comparisons.





Separate the two comparisons of each decision box to obtain the decision tree in the left figure.

II. How to construct the optimal tree

Huffman algorithm (Stump)

(1) Given a set of n weights {w1, w2,..., wn}, construct n binary trees F= {T1, T2,..., Tn}, where each Ti contains only one root node with weight Wi, and both its left and right subtrees are empty.

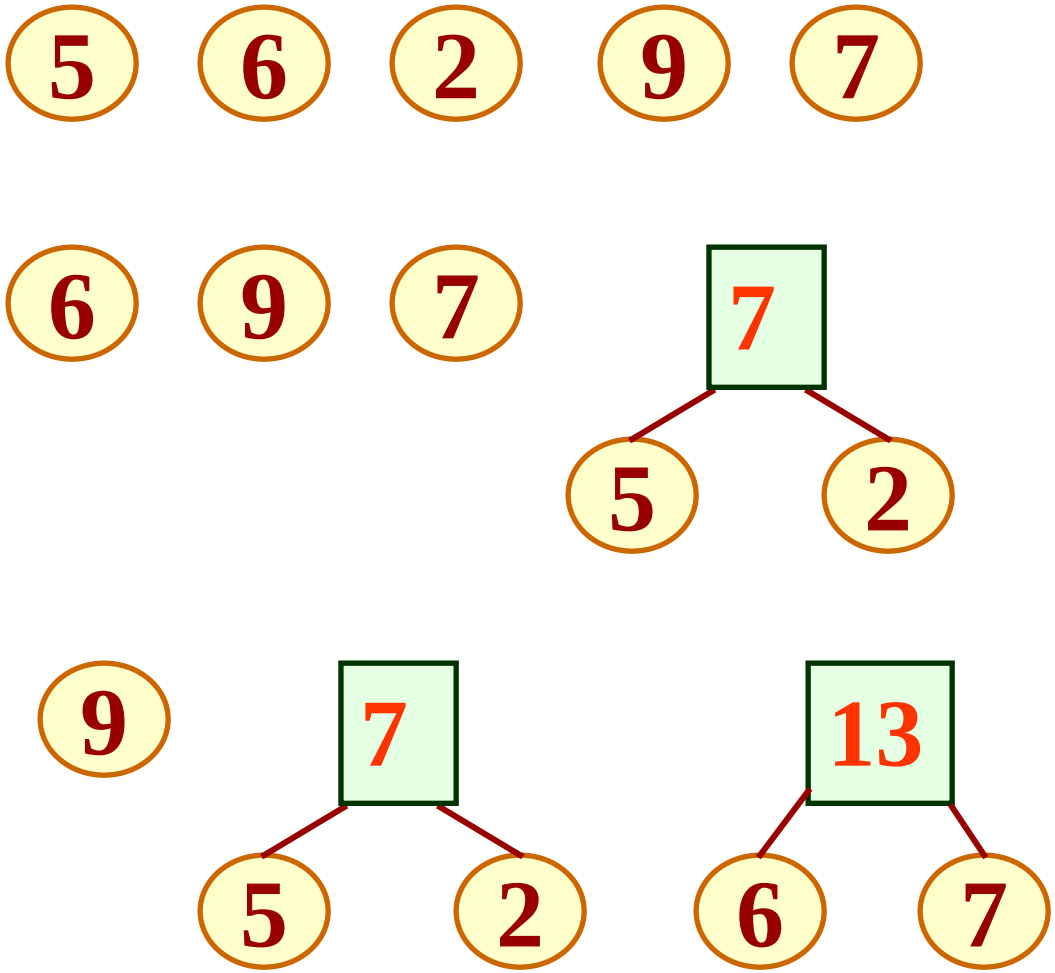
(2) Select two trees with the smallest root node weights in F as left and right subtrees to construct a new binary tree, and set the root node weight of the new binary tree as the sum of the root node weights of its left and right subtrees.

(3) Delete the two trees from F and add the resulting binary tree back to F;

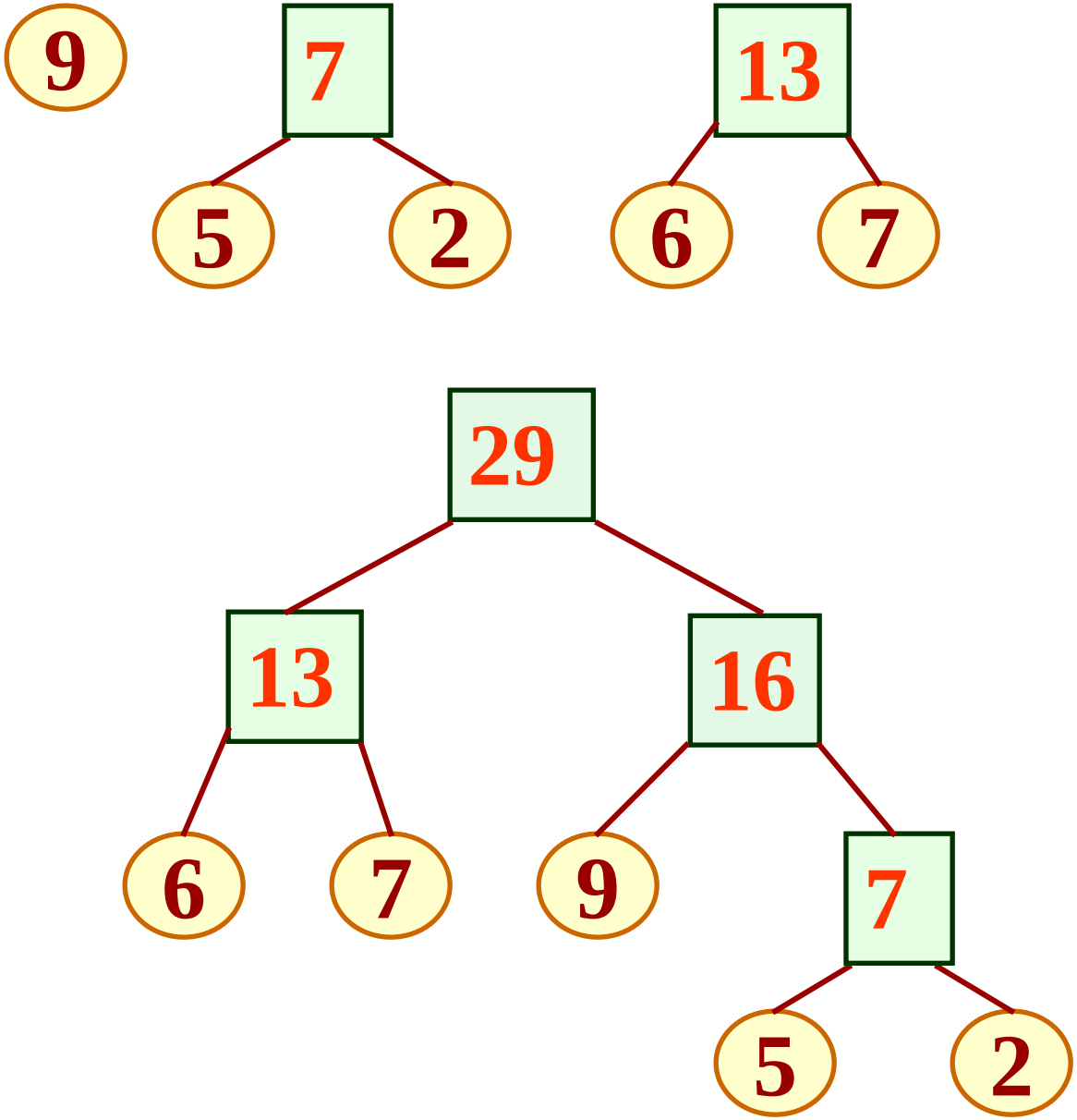
(4) Repeat steps (2) and (3) until F contains only one tree. This tree is the Huffman tree.

© Haiyang Liu, 2025

For example, the weights W are {5,6,2,9,7}.



© Haiyang Liu, 2025


9
7
5
2
13
6
7
29
13
16
6
7
9
7
5
2

3. Prefix Encoding

Message： ABACCDA

A:00
B:01
C:10
D:11

00 01 00 10 10 11 00

A B A C C D A

A:0
B:00
C:1
D:01

0 0 0 0 1 1 0 1 0

AAAA

B B

A B A

© Haiyang Liu, 2025

Prefix encoding

The encoding of any character is not a prefix of the encoding of another character in the same character set.

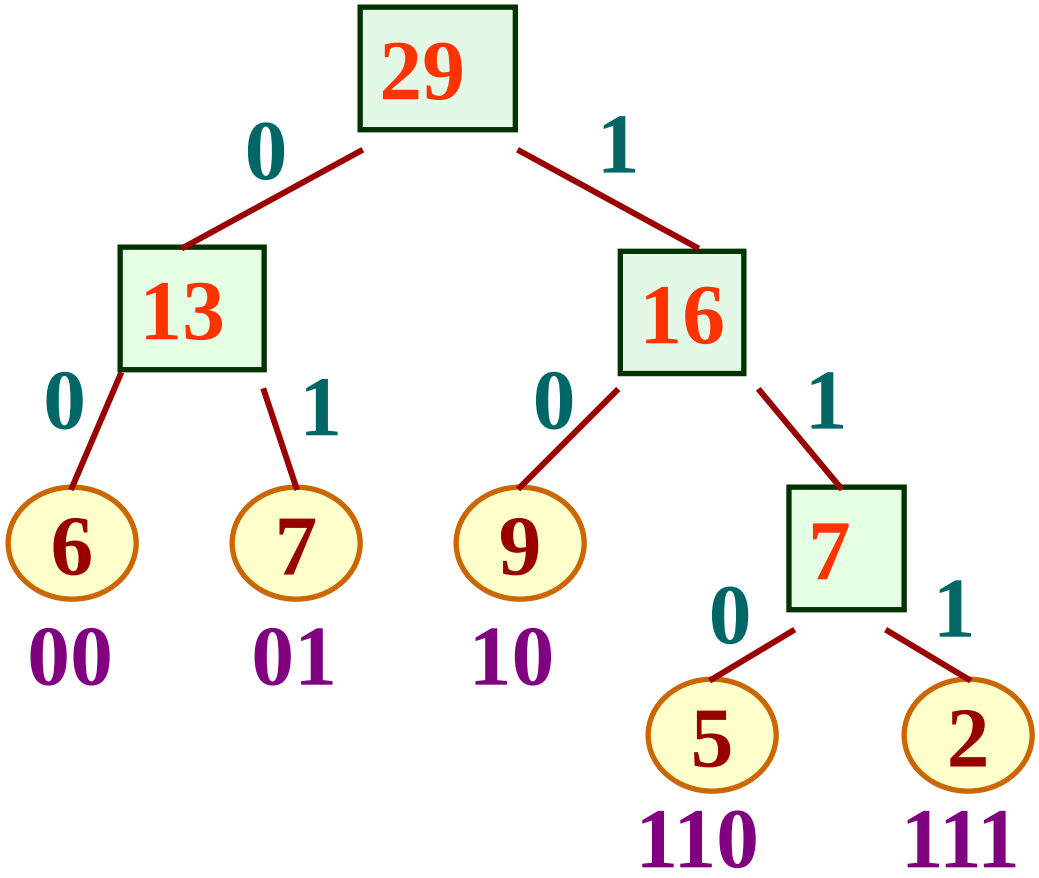
How to construct a prefix code?

Huffman trees can be used to construct a binary code of unequal length, and the Huffman code obtained through this construction is an optimal prefix code, which minimizes the total length of the transmitted message.

How do I construct a Huffman code?

Construction method:

The problem of designing a binary prefix code with the shortest total message length involves constructing a Huffman tree using the frequency of n characters as weights. By convention, the left branch represents character 0 and the right branch represents character 1. The path from the root node to a leaf node forms a string of branch characters, which serves as the code for that leaf node. This binary prefix code is known as a Huffman code.


29
0
1
13
16
0
1
0
1
6
7
9
7
00
01
10
0
1
5
2
110
111

Chapter 6 Summary of Tree Structure (1)

- Master the structural characteristics of binary tree and understand the corresponding proof methods.
- Familiar with the characteristics and application scope of various storage structures of binary tree.
- Traversing binary trees forms the foundation for all operations on them. The implementation of traversal algorithms depends on the storage structure employed. Mastering both recursive and non-recursive traversal strategies enables flexible application of these algorithms to perform other binary tree operations. Level traversal, on the other hand, follows a distinct search strategy.

Chapter 6 Summary of Tree Structure (2)

- The essence of binary tree threading lies in establishing direct connections between nodes and their predecessors/successors in the corresponding sequence. Proficiency in the threading process of binary trees and the method of locating a given nodes predecessor and successor in a pre-order threaded tree are essential. The threading process of binary trees is based on traversing the tree, while the threads in a threaded binary tree facilitate the corresponding traversal.

Chapter 6 Summary of Tree Structure (3)

- Be familiar with the storage structures of various trees and their characteristics, and master the conversion methods between trees, forests and binary trees. The establishment of storage structures is the prerequisite for other operations, so one or two methods of establishing binary trees and tree storage structures should be mastered.
- Learn to write algorithms that implement various operations of trees.
- Understand the characteristics of optimal tree, master the method of building optimal tree and Huffman coding.

Thank You !

School of Computer & Information Technology, Beijing Jiaotong University, Beijing, 100044, China